

DETECTION OF AI-GENERATED IMAGES USING DEEP LEARNING WITH EXPLAINABLE AI

*Report submitted to the SASTRA Deemed to be University
in partial fulfilment of the requirements for the award of
the degree of*

Bachelor of Technology

Submitted by

VENKATA KARTHIK (Reg. No.: 226003067, BTech-CSE)

KALIKI NARASIMHA (Reg. No.: 226003068, BTech-CSE)

SAKAM HARSHITH KUMAR REDDY (Reg. No.: 226003114, BTech-CSE)

May 2026



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SRINIVASA RAMANUJAN CENTRE
KUMBAKONAM, TAMIL NADU, INDIA – 612001**



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRINIVASA RAMANUJAN CENTRE

KUMBAKONAM, TAMIL NADU, INDIA – 612001

BONAFIDE CERTIFICATE

This is to certify that the project report titled "**Detection of AI-Generated Images Using Deep Learning With Explainable AI**" submitted in partial fulfilment of the requirements for the award of the degree of **B. Tech. Computer Science and Engineering** to the SASTRA Deemed to be University, is a bona-fide record of the work done by **Mr. VENKATA KARTHIK** (Reg. No. 226003067), **Mr. KALIKI NARASIMHA** (Reg. No. 226003068), **Mr. SAKAM HARSHITH KUMAR REDDY** (Reg. No. 226003114) during the final semester of the academic year 2025-26, in the Department of Computer Science and Engineering, Srinivasa Ramanujan Centre, under my supervision. This report has not formed the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

Signature of Project Supervisor :

Name with Affiliation : Dr S. Keerthana / AP-II / CSE

Date :

Project Viva voce held on

Examiner 1

Examiner 2



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

**SRINIVASA RAMANUJAN CENTRE
KUMBAKONAM – 612001**

DECLARATION

We declare that the report titled “**Detection of AI-Generated Images Using Deep Learning with Explainable AI**” submitted by us is an original work done by us under the guidance of Dr. S. Keerthana, Assistant Professor- II, Department of CSE, Srinivasa Ramanujan Centre, SASTRA Deemed to be University during the Eighth semester of the academic year 2025-26, in the Srinivasa Ramanujan Centre. The work is original and wherever we have used materials from other sources, we have given due credit and cited them in the text of the report. This report has not formed the basis for the award of any degree, diploma, associate-ship, fellowship or other similar title to any candidate of any university.

Name of Candidate(s)

Signature of Candidate(s)

Kadirimangalam Venkata Karthik

Kaliki Narasimha

Sakam Harshith Kumar Reddy

Date:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our Honorable Chancellor, **Prof. R. Sethuraman**, for providing us with the opportunity and the necessary infrastructure to carry out this project as part of our curriculum.

We are deeply thankful to our Honorable Vice-Chancellor, **Dr. S. Vaidhyasubramaniam**, and **Dr. S. Swaminathan**, Dean, Planning & Development, for their constant encouragement and strategic support throughout our college journey. We also extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University, for providing us with the opportunity to pursue this project.

Our heartfelt thanks go to **Dr. B. Santhi**, Dean, and **Dr. D. Narasimhan**, Associate Dean, and **Dr. A. Alli Rani**, Associate Dean, Srinivasa Ramanujan Centre, SASTRA Deemed to be University. We are also pleased to express our gratitude to **Dr. V. Kalaichelvi**, Associate Professor, Department of Computer Science and Engineering, Srinivasa Ramanujan Centre, for her encouragement and support during our project work.

Our guide, **Dr. S. Keerthana**, Assistant Professor-II, Department of Computer Science and Engineering, Srinivasa Ramanujan Centre, was the driving force behind this project from the very beginning. Her deep insights and invaluable suggestions guided us in making steady progress throughout the project. We also thank the project review panel members, **Dr. B. Sreedevi** and **Dr. S. Farjana Farvin**, for their valuable feedback and insights that significantly enhanced the quality of our work.

We would like to express our sincere gratitude to **Dr. Durga Karthik**, Assistant Professor-III, and **Dr. D. Vaishnavi**, Assistant Professor-II, Project Coordinators, Department of Computer Science and Engineering, Srinivasa Ramanujan Centre, for their invaluable guidance, continuous support, and encouragement towards the successful completion of our project report.

Finally, we gratefully acknowledge the unwavering encouragement and contributions from our families and friends, which played a vital role in the successful completion of this project. We thank you all for giving us the opportunity to showcase our skills through this project.

Table of Contents

Title	Page No.
Bona-fide Certificate	ii
Declaration	iii
Acknowledgments	iv
List of Figures	vi
List of Tables	vii
Abbreviations	viii
Notations	ix
Abstract	x
1. Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Literature Survey	2
1.4 System Requirements	3
1.5 Scope of the Project	4
1.6 Motivation	4
2. Objectives	6
3. Methodology and Experimental Work	7
3.1 Dataset preprocessing and Feature Extraction	7
3.2 Data Pipeline	10
3.3 System Architecture	10
3.4 Synopsis of the Algorithms Used	13
3.5 Model Training Configuration	16
3.6 Evaluation Metrics	17
4. Results and Discussions	19
4.1 Hybrid CNN Model Results	19
4.2 Efficientnet-B4 Model Results	22
4.3 Baseline CNN Model Results	24
4.4 Comparative Analysis	26
4.5 Explainability Analysis using GRAD-CAM	27
5. Conclusions and further work	29
5.1 Conclusions	29
5.2 Limitations	30
6. References	32
7. Appendix	33
7.1 Base Paper	33
7.2 Sample Source Code	45

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Architecture Diagram	13
4.1	Training and Test Accuracy Curves – Hybrid CNN	19
4.2	Confusion Matrix – Hybrid CNN Model	20
4.3	ROC Curve – Hybrid CNN Model	21
4.4	Training and Test Accuracy Curves – Efficientnet-B4 Model	22
4.5	Confusion Matrix – Efficientnet-B4 Model	22
4.6	ROC Curve – Efficientnet-B4 Model	23
4.7	Training and Test Accuracy Curves – Baseline CNN Model	24
4.8	Confusion Matrix – Baseline CNN Model	25
4.9	Grad-CAM Visualization for AI-Generated Image	27
4.10	Grad-CAM Visualization for Human (Real) Image	28

LIST OF TABLES

Table No.	Title	Page No.
1.1	Dataset split distribution	7
1.2	Dataset Statistics	8
3.4	Model Training Configuration	16
4.1	Hybrid CNN- performance metrics	21
4.2	Efficientnet-B4 CNN – Performance Metrics	23
4.3	Baseline CNN – Performance Metrics	26
4.6	Comparative Analysis of All Four Models	26

ABBREVIATIONS

<i>AI</i>	Artificial Intelligence
<i>CNN</i>	Convolutional Neural Network
<i>ELA</i>	Error Level Analysis
<i>RGB</i>	Red Green Blue (Image Format)
<i>DL</i>	Deep Learning
<i>TL</i>	Transfer Learning
<i>EfficientNet</i>	Family of Efficient Convolutional Neural Networks
<i>Grad-CAM</i>	Gradient-weighted Class Activation Mapping
<i>ROC-AUC</i>	Receiver Operating Characteristic – Area Under Curve
<i>TP</i>	True Positive
<i>FP</i>	False Positive
<i>TN</i>	True Negative
<i>FN</i>	False Negative
<i>ReLU</i>	Rectified Linear Unit (Activation Function)
<i>Sigmoid</i>	Sigmoid Activation Function
<i>TensorFlow</i>	Deep Learning Framework
<i>BatchNorm</i>	Batch Normalization
<i>PIL</i>	Python Imaging Library
<i>OpenCV</i>	Computer Vision Library
<i>F1-Score</i>	Harmonic Mean of Precision and Recall

NOTATIONS

x	Input RGB Image
x_{ela}	<i>ELA (Error Level Analysis) image</i>
y	Ground truth label (0: AI-generated, 1: Real)
$f_{rgb}(x)$	Feature representation extracted from RGB branch(EfficientNet)
$f_{ela}(x)$	Feature representation extracted from ELA branch (CNN)
F	Combined feature vector after fusion
W	Learnable weight parameters of the network
b	Bias term
σ	Activation function (ReLU / Sigmoid)
$p(y)$	Predicted probability of class
L	Loss function (Binary Cross-Entropy)
Acc	Accuracy metric
$Prec$	Precision metric
Rec	Recall metric
$F1$	F1-score metric
H	Grad-CAM heatmap

ABSTRACT

Due to the emergence of generative artificial intelligence and its rapid growth, highly realistic synthetic images have been created on a large scale. This poses a big threat as it is becoming increasingly difficult to differentiate such images from real images. For the purpose of combating this problem, this project is based on automatically detecting AI-generated images using deep learning and providing a visual interpretation of the decision-making process. A Convolutional Neural Networks (CNN) is used to classify the input image as real or AI generated. It is trained and evaluated on a publicly available dataset containing both real and synthetic images. Though many such methods provide good classification results, they are often black-box models with poor interpretability. To deal with this problem, the proposed model incorporates features of Explainable Artificial Intelligence (XAI), which is accomplished using Grad-CAM. These Grad CAM models provide significant insights into the patterns and features learned by the CNN, thereby explaining the importance of the process. The combined approach of the detection model and the explanation model provides significant practical applicability to the detection system for the identification of fake images, verification of image content, and forensic analysis. This research aims to distinguish between genuine artworks and synthetically generated images using a benchmark dataset.

Keywords: *AI-Generated Image Detection, Deep Learning, CNN, Artificial Intelligence, Explainable AI, Grad-CAM, Synthetic Image Detection, Digital Image Analysis.*

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Artificial Intelligence, or AI, has been rapidly advancing in image generation over recent years, and in particular, through the development of what are known as diffusion models that can now convincingly and artificially create art that is indistinguishable from art generated by humans. This development brings issues of authenticity, potential misinformation, and challenges to digital trust.

To address this challenge, existing approaches have adopted deep learning techniques, including the deployment of Deep Learning approaches, particularly Convolutional Neural Networks that aim to distinguish between human-generated images and those created by current AI. Most of the existing approaches, however, work as black boxes, thus returning a classification decision without revealing the underlying justifications for such a decision.

In our project, our goal is to build a system that can distinguish between AI generated images and natural human images. We employ Error Level Analysis (ELA) as a preprocessing step to reveal faint compression artefacts embedded in the images. We then use a CNN to extract features from these processed images to classify between the two types of artwork.

To increase understanding of the model's decisions, Grad-CAM was used as an Explainable AI (XAI) technique to create visualizations of the areas that affected the model's score most.

In our work, we aim to present an accurate and explainable deep learning approach for detecting artificially generated images. It can be used for a variety of tasks in digital forensics and content verification.

1.2 PROBLEM STATEMENT

Generating high quality images has evolved as a rapidly developing field in recent years with Generative AI, resulting in some images being so convincing they often look like they were created by humans themselves, making it increasingly difficult for people to identify real and fake images.

Artificially generated images are everywhere – this is great for entertainment purposes such as movies and games. Yet there are some very serious and important aspects to consider like is this going to increase misinformation and/or fake news, who owns the rights to something generated by AI, and will programs be able to recognize “fakes” to help aid digital crime scene investigations in the near future? If we are to tackle all of these challenges, the first step is to have an adequate image forensic system that can successfully detect forged images.

Most state-of-the-art deep learning solutions currently utilising deep neural networks to classify AI generated images exist as so called black-boxes: meaning although they have high accuracy at image classification they lack any kind of understandability or explainability for their given results, which currently means that users do not have much trust in these models and as a result cannot apply them in practical real world scenarios.

For rigorous identification of AI-generated content, an accurate and explainable image detection system is needed; this requires an ability to visually explain its decisions. Using a suite of Explainable AI (XAI) techniques provides one way to increase interpretability, improve trust, and product a more reliable and practical tool for image forensics and content verification tasks.

1.3 LITERATURE SURVEY

In recent years, various approaches based on deep learning and feature-based methods have been applied to the problem of detecting AI-generated images.

Yavuz Selim Taspinar (2024) proposed a deep learning architecture for fake image detection based on the usage of pre-trained versions of popular CNN models with a technique known as transfer learning. This approach is very simple to implement, has very good feature extraction capabilities and, as a result, achieves very competitive performance results on the images in which it is trained. However, it has very low generalization capabilities to novel generative models and most importantly, it is opaque in terms of understanding of its decisions, merely working as a black-box.

In Bird and Lotfi (2023), CIFAKE videos and frames were introduced and addressed using a CNN-based binary classifier and backed up with Grad-CAM explanations, where the model performed well on classification as well as provided insights into the decisions. However, the model failed to generalise to generated images (from outside the training distribution) quite severely.

In Francesco Cozzolino, et al., Multi-Modal Detection (MMD) Detectors based on CLIP features combine strengths of Image Models and Language Models achieving state of the art accuracy levels and semantic understanding. Nevertheless, their computational complexity prevent their real-time deployments.

Zheng et al. (2024) present a patch-based CNN denocator that breaks semantic artifacts in images generated by AI models. The method is evaluated on several different generative models including GANs and diffusion models. The method is very powerful but requires a complicated training pipeline and high computational cost.

In Yang et al. (2025), the authors explored frequency-domain and texture-based features to detect deepfakes, AI-generated images. They report high accuracy but their approach requires a lot of preprocessing and does not robustly handle image compression.

In Yumlembam et al. (2025), a system was designed that combined several uncertainty measures for AI image detection. This approach provided an effective means of pruning out incorrect classifications and achieving improved performance on untested data. However, the resultant model increased in terms of complexity and requires the manual adjustment of multiple components.

Although existing approaches based on ELA achieved high accuracy in natural language processing tasks, they often failed to generalise to unseen models and, crucially, lacked interpretability. This project goal is to improve the both performance and interpretability of existing approaches by combining ELA-based preprocessing with a CNN text classifier and the Grad CAM technique for explainability.

1.4 SYSTEM REQUIREMENTS

1.4.1 Hardware Requirements

- **Processor:** A system equipped with an Intel i5 / AMD Ryzen 5
- **RAM:** Minimum 8 GB (16 GB recommended for improved performance)
- **Storage space:** At least 10 GB of available storage space
- **GPU:** GPU support such as Google Colab GPU (T4) for faster training
- **System Type:** 64-bit system

1.4.2 Software Requirements

- **Operating System:** using Windows / Linux / macOS
- **Programming Language:** using Python (version 3.8 or above)
- **Development Environment:** Google Colab
- **Libraries used:**
 - Tensor Flow / keras for deep learning
 - NumPy for numerical operations
 - OpenCV for image processing
 - Matplotlib for visualization
 - Scikit learn for data splitting & evaluation
- **Tools Used:**
 - Google Colab (model training with GPU)
 - Kaggle (dataset source)

1.5 SCOPE OF THE PROJECT

This project is exploring the possibility of building a computer vision system that can tell if an image was generated by a human or a computer. The system would use deep learning techniques for classifying images as either human or AI generated. It would integrate modern techniques in deep learning area with those in image forensics. The system would be designed to deal with the increasing problem of distinguishing between real and synthetic images.

This project uses a large dataset of over 25000 images from various artistic styles – both human and AI generated – and builds on the work of compressing images and utilising Error Level Analysis (ELA) to highlight subtle compression errors in the images. The preprocessed images are given as input into a Convolutional Neural Network to train the network to distinguish between compressed and uncompressed images for both human and AI images.

To increase the model's understanding, Explainable AI methods like Grad CAM have been integrated into the system to make the predictions more transparent and explainable, which is particularly relevant for real-world use cases and has huge potential to make models more trustworthy.

Our project also involves a comparison of approaches for detection such as vanilla CNN-based approaches and explainable approaches. We have made a simple GUI which can be used to upload images and get results along with explanations in real time.

The system's performance is measured using the accuracy and the validation performance. The system's ability to generalise to new images, created by AI, will also be evaluated. The goal of the system is to achieve high detection accuracy and show good generalization capabilities.

It is important to note that this project is focused on image classification. It is not dealing with video-based deepfake detection or any large-scale, real-time applications. Consequently, the results obtained may not generalise to images that have been heavily compressed or otherwise modified from their original form, even if found on the web and in public datasets.

This project forms a basis for future work on constructing robust and explainable AI-based image detection tools that can be used in variety of fields including digital forensics, media verification, and ethical AI.

1.6 MOTIVATION

The internet is awash with Images Generated by AI (Deepfakes, specifically) that are becoming increasingly realistic and far more prevalent; it is no longer sufficient for human eyes and brains to simply tell the difference between “genuine and real” versus “artificial and fake”, as new models such as diffusion-based models are able to generate images indistinguishable from human-generated ones.

Existing image detection techniques generally focus on visible features and simple patterns, which are insufficient for uncovering the often subtle artefacts that are generated by state of the art AI models. Most current deep learning-based image detection methods are black-box solutions that are able to achieve high accuracy but which provide little insight into how

decisions are made, thus leaving a large trust gap for applications such as digital forensic analysis, media verification, and content authenticity.

This challenge raises the bar to not only an accurate detection of deep learning generated images but also an accompanying explanation of the decisions made by the model. By incorporating Error Level Analysis (ELA) into an image forensics solution, hidden compression artifacts and inconsistencies can be revealed which in turn enable a deep learning model to learn features which are more meaningful than just looking at the images' appearance.

We also implemented Explainable AI methods like Grad CAM to allow for the visualization of regions the system focuses to make a decision and assesses whether the model is paying attention to informative or spurious features.

We combine techniques from deep learning, image forensics and explainability to develop a transparent, robust and accurate detection approach for DOS attacks.

CHAPTER 2

OBJECTIVES

We are working on a project that aims at building a deep learning-based system capable of classifying images with high accuracy and explainability. As generative models are getting better at generating human-looking images, there is an ever-increasing need for developing systems that can detect whether an image was generated by an AI or by a human.

The primary goal of this research involves using CNN to learn such complex visual, hierarchical representations that can distinguish between real images and images generated by a computer. Deep learning is used to automatically learn features of images and classify images at a depth within the network that manually extracting features from images cannot recognize.

In addition to the main objectives, we aim to utilise Error Level Analysis (ELA) as a preprocessing technique to reveal concealed or imperceptible characteristics of hidden data within images, with a view to improve detection of AI-generated content that looks realistic but diverges in structure from naturally generated images.

In order to reach the best detection results, we plan to implement a hybrid network that merges the RGB features with the features obtained from ELA in order to exploit semantic information and artefact-specific features characterizing different images and in order to improve the generalization capabilities.

To increase the trustworthiness of the proposed architecture for content classification and authentication, the goal of this part of the project is to integrate techniques for the explainability of AI (XAI), and in particular to use Score-CAM to generate visual explanations of deep learning models' decisions. Providing information about the parts of the image that account for the largest portion of the score for the assigned class increases the interpretability of the system. XAI will facilitate both digital forensics and content verification.

The project aims to also implement and test different models to recognize the most suitable one to use in the project. The implemented models include a hybrid CNN model, a transfer learning model based on EfficientNet, and a baseline CNN model to compare the results and to understand which method is the most effective for the project's goals.

We also aim to evaluate the system on evaluation metrics such as accuracy, validation performance and generalisation performance on images generated by new, never before seen generative models. This makes sure that the system is not only performing well on training data, but is also able to handle unseen data.

Our goal with this project is to create an accurate and scalable approach to finding images generated by AI, with usage in digital forensics, media authentication and more—to make sure that AI is developed and used ethically.

CHAPTER 3

METHODOLOGY

3.1 DATASET PREPROCESSING AND FEATURE EXTRACTION

3.1.1 Dataset Description and Organization

In this project, we use the AI-ArtBench Dataset (Dataset) that contains a collection of 2 categories of art: human/physical and digitally produced by artificial intelligence (AI). This dataset is made up of 185,015 total images - 60,000 human generated images and 125,015 AI-generated images using the following models: Latent Diffusion and Standard Diffusion.

The AI-ArtBench Dataset contains ten different artistic styles (e.g. Impressionism, Realism, Surrealism, Renaissance, Ukiyo-e) in order to create a sufficient mix of visual patterns. A diverse dataset like this will allow the model to generalise across a number of different styles; otherwise, the model would overfit to an individual category/style.

The methodology to assess performance and generalisation is to structure the dataset:

- **Training Dataset:** Human + Latent Diffusion images
- **Validation Dataset:** Used for hyperparameter tuning
- **Testing Dataset:** Used for final evaluation

This separation ensures that the model is tested on **unseen generative techniques**, making the evaluation more realistic.

Dataset Split Distribution

Table 1.1 – Dataset Split Distribution

Dataset Split	Human Images	AI Images (LD)	Total Images
Training	~35000	~36000	~71000
Validation	~7500	~7800	~15300
Testing	~7500	~7800	~15300

Table 1.2 - Dataset Statistics

Parameter	Value
Dataset Name	AI-ArtBench
Total Images	185015
Human-Created Images (Total)	60000
AI-Generated Images	125015
└ Latent Diffusion	60000
└ Standard Diffusion	65015
Number of Classes	2(Human, AI)
Image Size	256*256*3
Artistic Styles	10 Categories
Training Data Split	70%
Validation Data Split	15%
Test Data Split	15%

3.1.2 Image Pre-processing

Image pre-processing plays a vital role in increasing the performance and ability of models in deep learning. In this project, multiple required preprocessing steps are performed to standardize the input data and enhance differentiating features required for finding AI generated images:

a) Image Resizing

All the images present in the dataset are resized to 256×256 pixels to make sure and set a uniform input size for the CNN model. This helps in reducing computational costs while holding important visual information.

b) Color Space Conversion

In this process, all the images are converted from BGR to RGB format, because OpenCV loads images in BGR format while deep learning models require RGB format. This makes sure correct color representation for feature extraction.

c) Pixel Normalization

In this process, value of each pixel is scaled from the range of 0-255 to 0-1. Normalizing the pixel values is important because it provides better stability for training the model and helps the model to learn fast during the training.

d) Error Level Analysis (ELA)

Error Level Analysis is an important preprocessing step that is used to mark the compression artifacts.

- The image is recompressed at **90% JPEG quality**
- The difference between original and compressed image is computed
- The difference is amplified to enhance visibility Observation:
- AI images → uniform patterns
- Human images → irregular patterns

ELA helps the model to detect micro details that are not visible to the human eye.

e) Final Input Preparation

After performing all pre-processing steps, the images are converted into array-formatted images of shape $(256 \times 256 \times 3)$ appropriate for suitable model input.

3.1.3 Data Splitting

To handle class imbalance, the dataset is split using stratified sampling into:

- **70% Training**
- **15% Validation**
- **15% Testing**

Stratification sampling ensures equal representation of both the label classes present in the dataset. (AI label and Human label).

3.2 DATA PIPELINE

The tf.data API is used for the Data Pipeline in order to create a TensorFlow-based solution that is fast to load data through the pipeline.

The processing stages of the pipeline include:

- Dynamic image loading from file paths
- Preprocessing (ELA, resizing, normalization)
- Batch formation (batch size = 32)
- Prefetching for improved training speed

This pipeline ensures efficient utilization of computational resources and faster training.

3.3 SYSTEM ARCHITECTURE

The proposed system is a multi-stage pipeline utilizing a combination of AI-based image forensics techniques to detect digital artwork that was created using AI. It is comprised of three key components; Preprocessing, Deep Learning Model Processing, and Explainable AI Processing to ensure that results are accurate and easily interpretable.

3.3.1 Input Layer

The user of the system will enter an input image as part of the systems interface through either a user-interface or dataset. Examples of the input may be any format of digital artwork; JPEG or PNG, etc. This allows the user to utilize both datasets and global internet images as input to the system.

3.3.2 Preprocessing Layer

This stage processes the input image to achieve a standardized and enhanced feature set:

- Resizing: All images are resized to a fixed size (256×256)
- Normalization: All pixel values will be normalized to the range of $[0, 1]$

- Error Level Analysis (ELA): This method will generate a secondary image for analysis

The Result of the ELA process helps to detect AI generated images through producing a representation of inconsistencies due to compression between two images. Therefore, this stage provides for two parallel processed images to enter into the next stage of the pipeline:

- An RGB image representation of the original image.
- An ELA representation of the original.

3.3.3 Feature Extraction Layer

The feature extraction layer facilitates the extraction of descriptive features that can be used for model training based on both inputs in parallel.

- Features used to train the model for RGB images as well as high-level semantic features such as Shapes, Textures and Patterns will be drawn from a pre-trained EfficientNet.
- Low-level artifacts and inconsistencies associated with image compression will be drawn from an ELA image using a custom CNN.

With this approach to architecture, the system will be able to learn two distinct types of information about an image.

- Visual content
- Forensic content

3.3.4 Feature Fusion Layer

The feature fusion layer integrates the outputs of the features extracted from the RGB and ELA images through the use of concatenation. The result of this is a representation that contains both semantic the forensic features. The final features used for classification will pass through the following processing functions:

- Use of batch normalization to stabilize the training of the model.
- Use of fully connected (dense) layers to learn complex relationships in the data.
- Use of dropout to reduce the risk of overfitting the model.

The combined nature of the different types of features (i.e., semantic and forensics) will improve the classification accuracy of the model once completed.

3.3.5 Classification Layer

To obtain a final “output” value between 0 and 1, the combined features of both inputs are fed to a sigmoid activation level.

- If the output is very low (near 0), then it can be concluded that this image is created by a human.
- If the output is very high (near 1) then it can be determined that this image was created by an artificial intelligence (AI).

Binary layer classification is performed by this layer based on learnt patterns during model training.

3.3.6 Explainability Layer (Grad-CAM)

The explainability of the system is achieved through the use of Grad-CAM as follows:

- Using extracted feature maps from the last convolutional layer.
- Each of the feature maps acts as a mask on the input image.
- The model’s prediction score is determined for each of the masked input image(s).
- By taking a weighted combination of all of the feature maps, a heatmap is generated to show which regions within the input image contributed to generating the final predicted class.

Ultimately this heatmap shows the location(s) in the input image that contributed to the specific predicted class (AI vs Human).

3.3.7 Output Layer

In summary, when an image has been processed/analysed through the Explainable AI system, the following information will be available at the output of the system:

- Predicted class (AI or Human)
- Confidence score
- ELA image
- Heatmap visualization
- Overlay of heatmap on original image

The output information generated by the systems will all be presented via a GUI and will establish user-friendly interactivity.

3.3.8 System Flow Summary

The entire workflow process of the Explainable AI system can be summarized using the flow diagram shown previously:

Input Image → Preprocessing (Edge Level Analysis + Normalising) → Feature Extraction (RGB Feature + ELA Feature Extraction) → Fused Features → Predict Class → Score-CAM → Output Functionality (View Output Results via user graphics interface)

3.3.9 Architecture Diagram

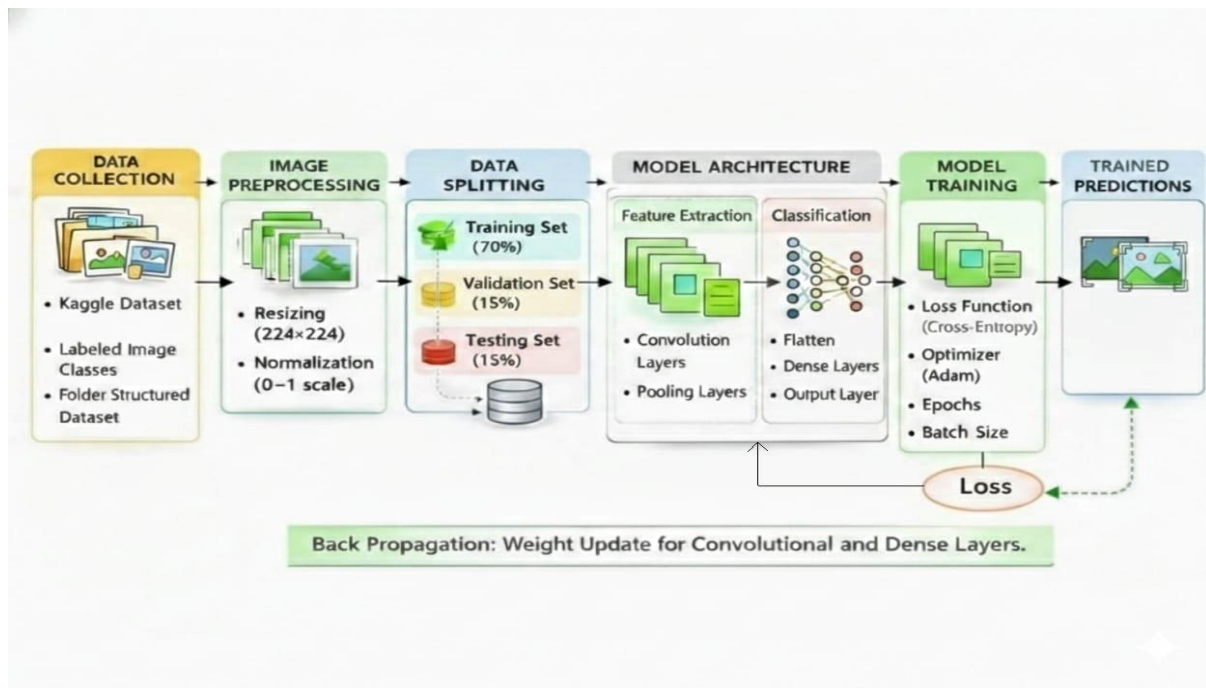


Fig.3.1. Architecture Diagram

3.4 SYNOPSIS OF THE ALGORITHMS USED

3.4.1 Convolutional Neural Networks (CNN)

CNN is a deep learning model designed to process data with a grid like topology such as images. CNN is used in the project because of image classification.

Working Principle:

A CNN processes images with processing filters through convolution operations to detect patterns within an image. The architecture consists of:

- Convolution Layers → Extract spatial characteristics of an image such as edge or texture
- Activation Functions (ReLU) → Introduce non-linear characteristics in the neural network

- Pooling Layers → Reduces the image's spatial resolution while retaining needed characteristics
- Fully Connected Layers → Classification of the object in an image

Logic in this Project:

CNN is used to:

- Learn hierarchical features from ELA images
- Capture compression artifacts
- Distinguish between structured (AI) and irregular (human) patterns

3.4.2 EfficientNet (Transfer Learning Model)

EfficientNet is the latest CNN architecture optimally used by scaling depth, width and resolution.

Working Principle:

EfficientNet uses compound scaling:

- Depth → Total number of layers
- Width → Total number of channels
- Resolution → Size of the Input image

The compound scaling of EfficientNet increases computer vision performance without creating a significant increase in the computational cost of running a neural network.

Logic in this Project:

- EfficientNetB0 is used in the hybrid model for RGB feature extraction
- EfficientNetB4 is used as a standalone model for benchmarking

These models capture:

- High-level semantic features
- Artistic patterns and structures

3.4.3 Feature Fusion Technique

The feature fusion combines multiple feature representations into one vector.

Working Principle:

- Features from RGB and ELA branches are concatenated

- Dense layers learn relationships between combined features

Logic in this Project:

- The RGB features capture the visual semantics of the image.
- The ELA features capture the compression artifacts of the image.

Combining both together can improve classification accuracy and robustness.

3.4.4 Grad-CAM (Explainable AI)

Grad-CAM is technique that produces visual heatmaps, highlighting the crucial image regions a Convolutional Neural Network (CNN) uses to make predictions.

Working Principle:

1. Extract feature maps from the last convolution layer
2. Normalize and resize each feature map
3. Use each feature map as a mask on the input image
4. Compute prediction scores for masked images
5. Generate heatmap using weighted combination

Mathematical Representation:

Heatmap is computed as:

$$N \text{ Heatmap} = \sum_{i=1} S_i F_i$$

Where:

- S_1 = prediction score for masked image
- F_1 = feature map

Logic in this Project:

- AI images → show large concentrated regions
- Human images → show scattered patterns

Score-CAM provides:

- Better stability than Grad-CAM
- More reliable explanations

3.4.5 Binary Classification using Sigmoid

The final classification is performed using a sigmoid activation function.

Working Principle:

$$\sigma(x) = 1/(1+e^{-x})$$

- Outputs probability between 0 and 1
- Threshold used to classify AI vs Human

Logic in this Project:

- Score > 0.5 → AI-generated
- Score < 0.5 → Human-created

3.5 MODEL TRAINING CONFIGURATION

In this project, we implemented three different models, Hybrid CNN (RGB + ELA), EfficientNet-B4 and Baseline CNN. We then trained and evaluated the three models using a pipeline. The dataset is divided into three parts: training, validation and test sets. We used stratified split, meaning that in each set, the class distribution was maintained. Hybrid CNN and EfficientNet models were trained using the complete dataset, while Baseline CNN was trained on a much smaller subset for comparison purposes.

Table 3.4 Model Training Configuration

Parameter	Hybrid CNN (RGB + ELA)	EfficientNet-B4	Baseline CNN
Dataset Split	80% Train / 20% Val	80% Train / 20% Val	80% Train / 20% Val
Input Size	224 × 224 × 3 (RGB + ELA)	224 × 224 × 3	224 × 224 × 3
Optimization Algorithm	Adam Optimizer	Adam Optimizer	Adam Optimizer
Learning Rate	1e-4 → 1e-5 (fine-tune)	1e-4 → 1e-5 (fine-tune)	0.001
Loss Function	Binary Crossentropy	CrossEntropyLoss	Binary Crossentropy

Pretrained Weights	EfficientNet-B0 (frozen → unfrozen)	Yes (EffNet-B4)	No
Regularization	Dropout (0.5), BatchNorm	Implicit (pretrained)	EarlyStopping, LR Reduce

3.6 EVALUATION METRICS

To assess the proposed models, a thorough set of evaluation metrics are utilized. Since the classification process is binary, i.e., images are classified between AI-generated and human-created images, the chosen metrics reflect different aspects of the classification performance metrics. The performance of the proposed models is assessed using the test images to verify their ability to generalize.

Accuracy:

Accuracy gives the proportion of correctly classified predictions out of the total number of predictions. It provides an overall measure of model performance.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

Precision tells you the number of positive cases predicted correctly as a percentage of the total positive cases predicted. A highest precision value will indicate the lowest number of false predictions for positive cases.

$$Precision = \frac{TP}{TP + FP}$$

Recall(Sensitivity):

Recall (or sensitivity) is used to measure the total number of actual positive samples that the model has predicted correctly. Thus, it is a good indication of the ability of the model to find all correct samples.

$$Recall = \frac{TP}{TP + FN}$$

Specificity:

Specificity indicates how many cases were predicted by the model correctly as negative. A high specificity value indicates that the model has verified that some cases were not true positives before making its predictions.

$$Specificity = \frac{TN}{TN + FP}$$

F1-Score:

The F1-score is a derived number based on the precision and recall numbers. Since F1-score is a harmonic mean of both numbers, it can be seen as a value that provides the average performance of your model when a high score on false positives is balanced against a low score on false negatives.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

ROC-AUC:

ROC – AUC measures the ability of a model to discriminate between the image possibilities from AI vs. Human by measuring the model's confidence levels over all the possible classification thresholds for each observation. The larger the AUC, the higher the performance in discriminating between the AI images & Human images.

Confusion Matrix:

The confusion matrix is a 2×2 matrix that is used to visualize classification performance of the model. Rows represent actual classes, while columns represent predicted classes. The diagonal elements indicate correct predictions, while off-diagonal elements represent misclassifications. It provides detailed insight into how well the model distinguishes between AI-generated and human images.

Where TP = True Positive, TN = True Negative, FP = False Positive, and FN = False Negative.

CHAPTER 4

RESULTS & DISCUSSION

In this section, experimental outcomes of the three developed models are briefed. The results of Hybrid CNN model (EfficientNet-B0 with ELA feature fusion), EfficientNet-B4 model and baseline CNN model are evaluated based on accuracy, precision, recall, F1-score, ROC-AUC and confusion matrix plots for classification task of distinguishing images generated by AI and images created by humans. Training and validation results obtained from experimental runs on images in the dataset are reported alongside the results obtained while testing the models on additional images samples. A comparative study is performed to discuss the effectiveness and limitations of developed models and to identify which model performed best in images generated by AI.

4.1 HYBRID CNN MODEL RESULTS

4.1.1 Training Performance

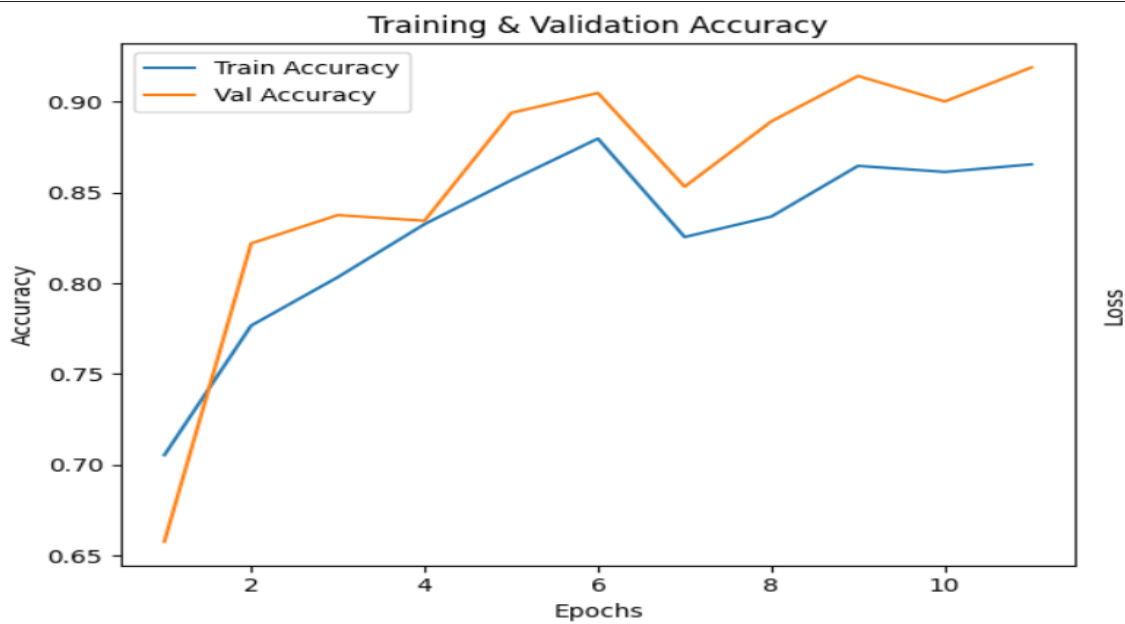


Fig. 4.1. Training and Test Accuracy Curves – Hybrid CNN Model

The Hybrid CNN is trained in two phases: in the initial training phase and in the fine-tuning phase. In the initial training phase, the network was trained for 6 epochs in which the training accuracy and the validation accuracy both increased. The training accuracy increased from 70.5% up to a maximum of 88.0%, while the validation accuracy increased from 65.7% up to a maximum of 90.1%.

During evaluation of the performance of the model with all learning rates unfrozen for the fine-tuning phase, I also managed to improve the feature learning and overall validation accuracy of the upper layers. The top layers of EfficientNet-B0 were unfreeze and with some tuning the model reached an accuracy of 91.72%.

There is only a little difference between the training error and the cross-validation error; this does not indicate significant overfitting. Accurate on test: 92.43%.

4.1.2 Visualization of Evaluation Metrics

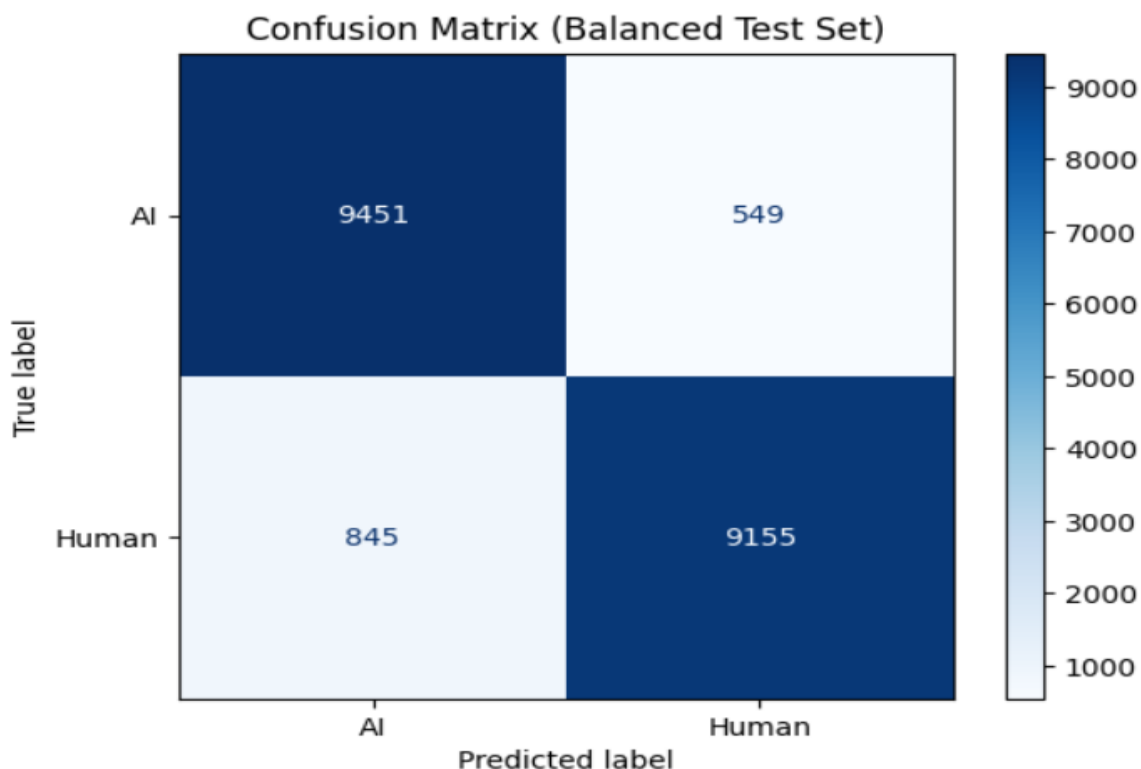


Fig. 4.2. Confusion Matrix – Hybrid CNN Model

From the confusion matrix, it can be observed that:

- The model correctly classifies a large number of both AI and human images.
- The number of misclassifications is relatively low compared to correct predictions.
- Slight imbalance exists due to the dataset distribution, where AI samples are more dominant in the test set.

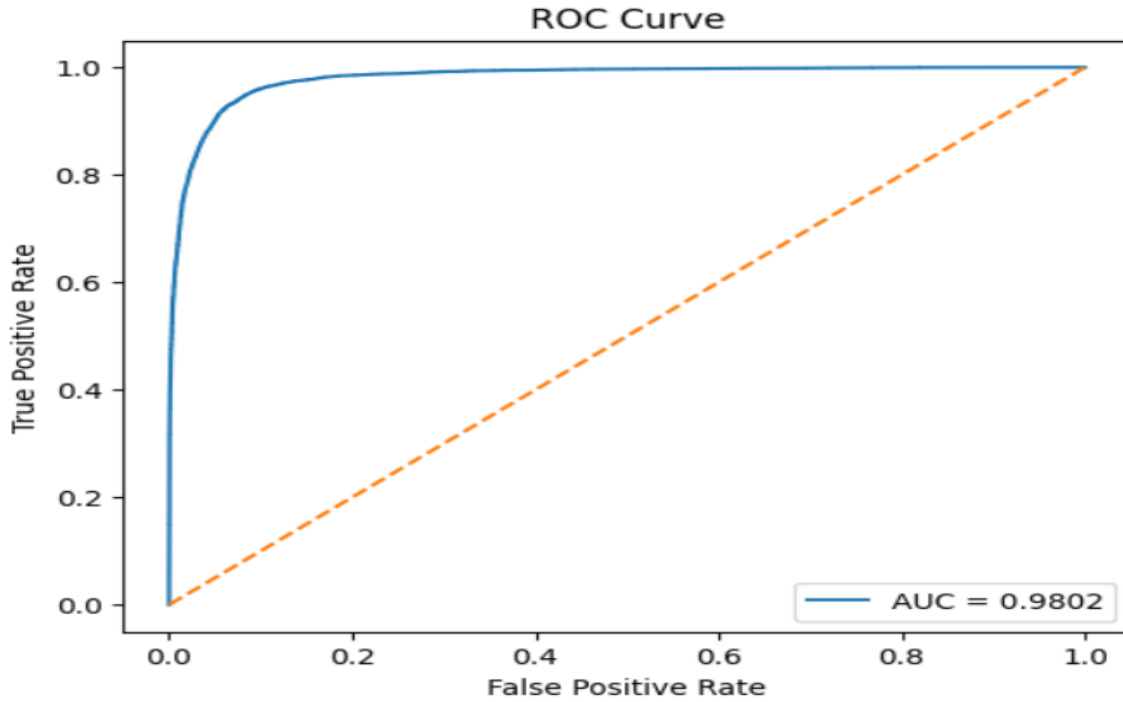


Fig. 4.3. ROC Curve – Hybrid CNN Model

Table 4.1 summarizes the general performance of the Hybrid CNN model:

Table 4.1 Hybrid CNN – Performance Metrics

Class	precision	Recall	F1-Score	Specificity	ROC-AUC
AI	0.9572	0.9441	0.9506	0.9155	0.9648
Human	0.8912	0.9155	0.9032	0.9441	0.9648

Our approach Hybrid CNN, uses both RGB and ELA features to classify images as human or AI generated and reports a strong test accuracy of 92.43%. The ROC-AUC score of 0.9648 reports a strong discriminative power. F1 score for the AI label is 0.9506, for the human label 0.9032. The specificity values are good and indicate a balanced performance. The small performance difference between the two classes might be due to the class imbalance in the used dataset and the high realism of generated images. Nevertheless, the results are consistent and reliable.

4.2 EFFICIENTNET-B4 MODEL RESULTS

4.2.1 Training Performance

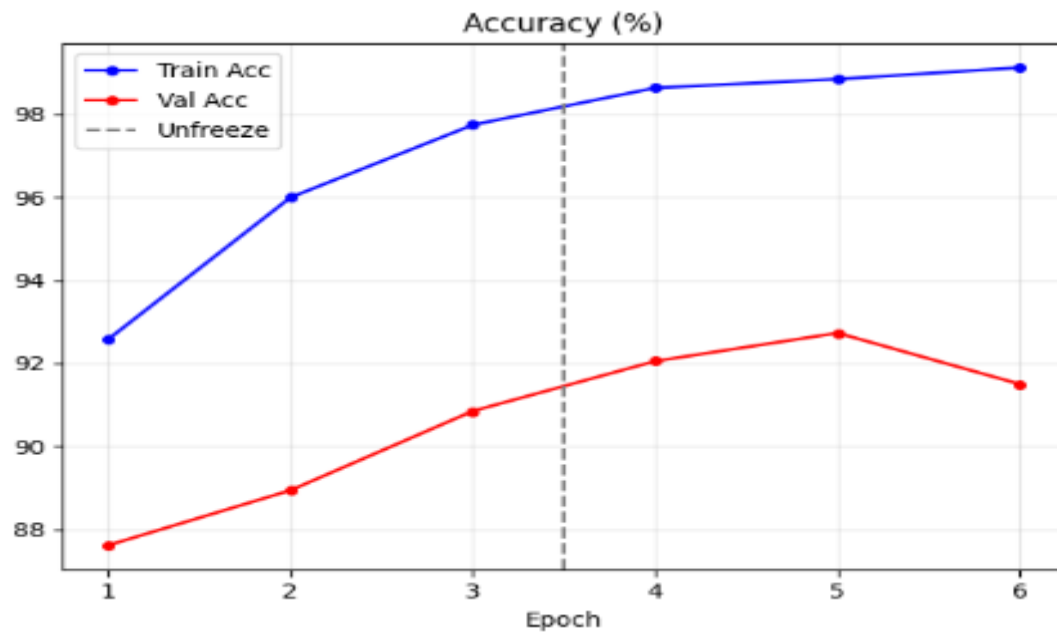


Fig. 4.4. Training and Test Accuracy Curves – Efficientnet-B4 Model

This is the EfficientNet-B4 model which was trained using transfer learning and then fine-tuned after it reached convergence. It reached about 99.1% training accuracy and 92–93% validation accuracy. The loss is consistent dropping for training and relatively flat for validation with slight overfitting after fine-tuning. Overall, the model demonstrates strong convergence and effective feature extraction capability.

4.2.2 Visualization of Evaluation Metrics

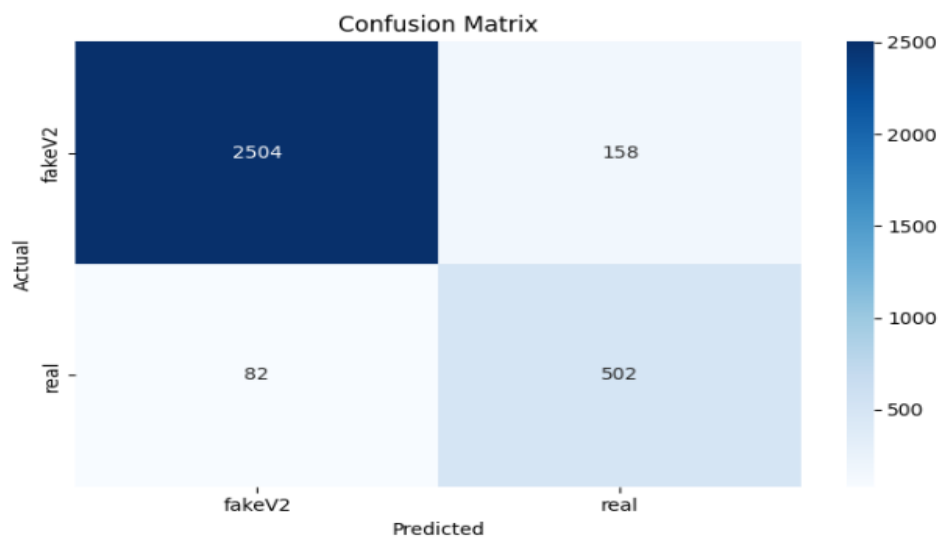


Fig. 4.5. Confusion Matrix – Efficientnet-B4 Model

The confusion matrix shows that the model correctly classifies the majority of AI images, with relatively few misclassifications. However, some real images are misclassified as AI, indicating slight bias toward the AI class.

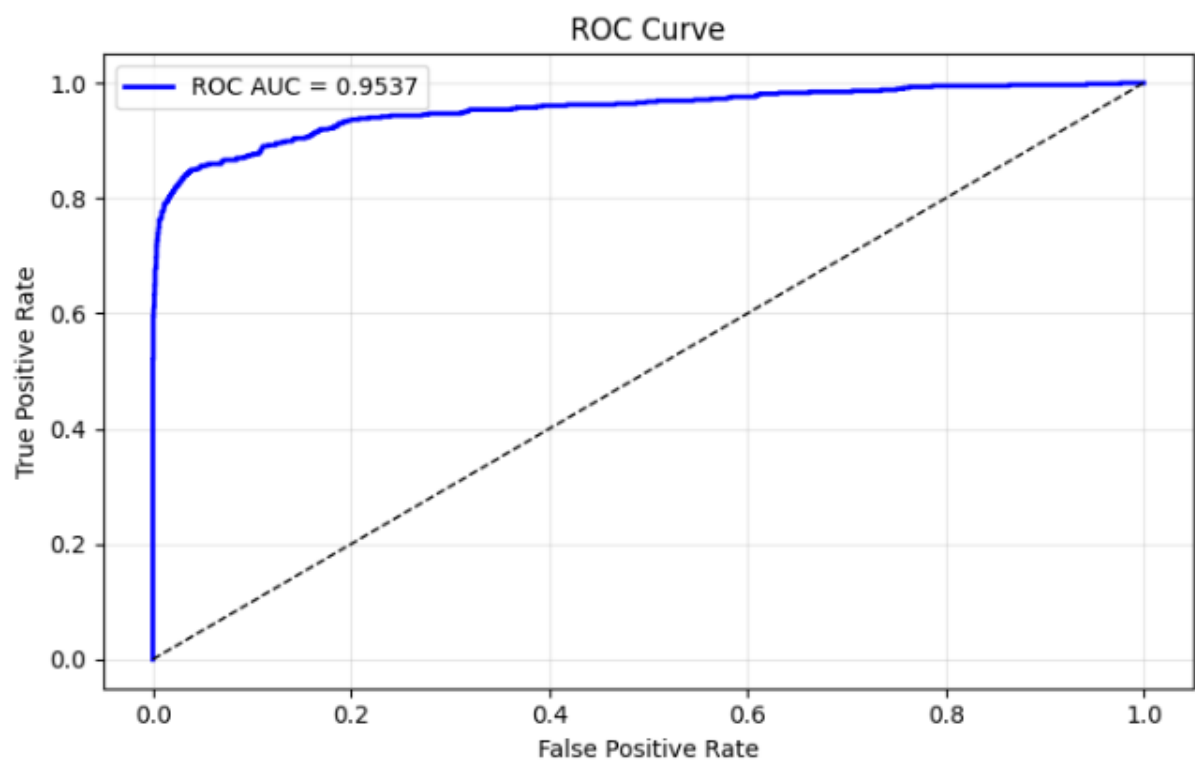


Fig. 4.6. ROC Curve – Efficientnet-B4 Model

Table 4.2 summarizes the general performance of the Efficientnet-B4 CNN model:

Table 4.2 Efficientnet-B4 CNN – Performance Metrics

Class	Precision	Recall	F1-Score	Specificity	ROC-AUC
AI	0.97	0.94	0.95	0.86	0.937
Human	0.76	0.86	0.81	0.94	0.937

The EfficientNet-B4 model obtained an accuracy of 93% with an overall ROC-AUC score of 0.9537. It specifically performed exceptionally well in the AI class with an F1 score of 0.95. However, its score in the human class was 0.81 which suggests it has room for improvement in real image detection. This can also be seen in the provided confusion matrix, where a considerable amount of real images were misclassified as being AI generated. This could be the result of the large class of AI images and the network's inherent bias towards this class due to dataset imbalance. Overall EfficientNet-B4 reached very high accuracy scores; however, they were not as evenly distributed as the scores reached by the Hybrid CNN model.

4.3 BASELINE CNN MODEL RESULTS

4.3.1 Training Performance

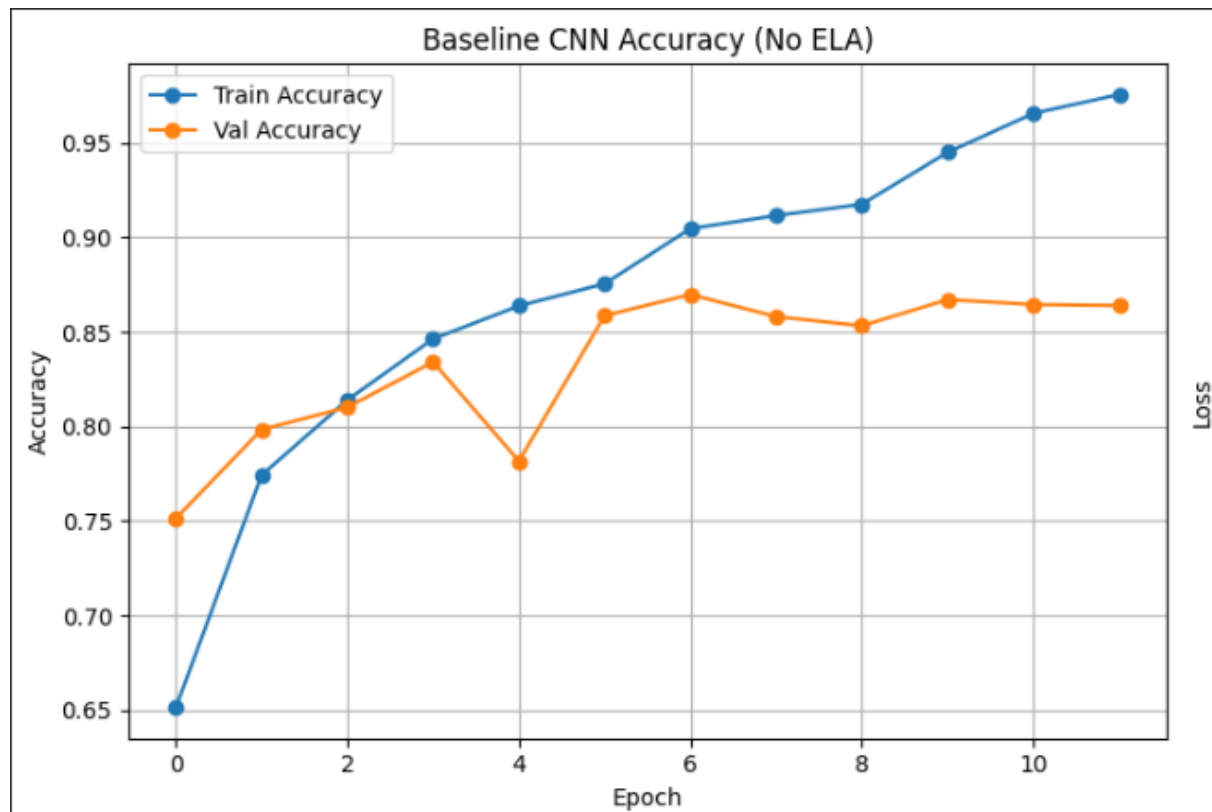


Fig. 4.7. Training and Test Accuracy Curves – Baseline CNN Model

We first trained the Baseline CNN on a reduced set of the images and did not utilise any ELA features. The training accuracy was steadily increasing from about 65% up to a correct classification rate of around 97%. On the other hand, the accuracy on the validation set only reached levels up to 86-87%. However, it did not reach a stable classification performance during the training run. Additionally, the loss values reveal overfitting as the training loss continued to decrease throughout the training, while the loss on the validation set varied and sometimes even increased during late epochs. This indicates that the model learned to merely

memorise the images from the training set rather than generalising also to images not seen before.

4.3.2 Visualization of Evaluation Metrics

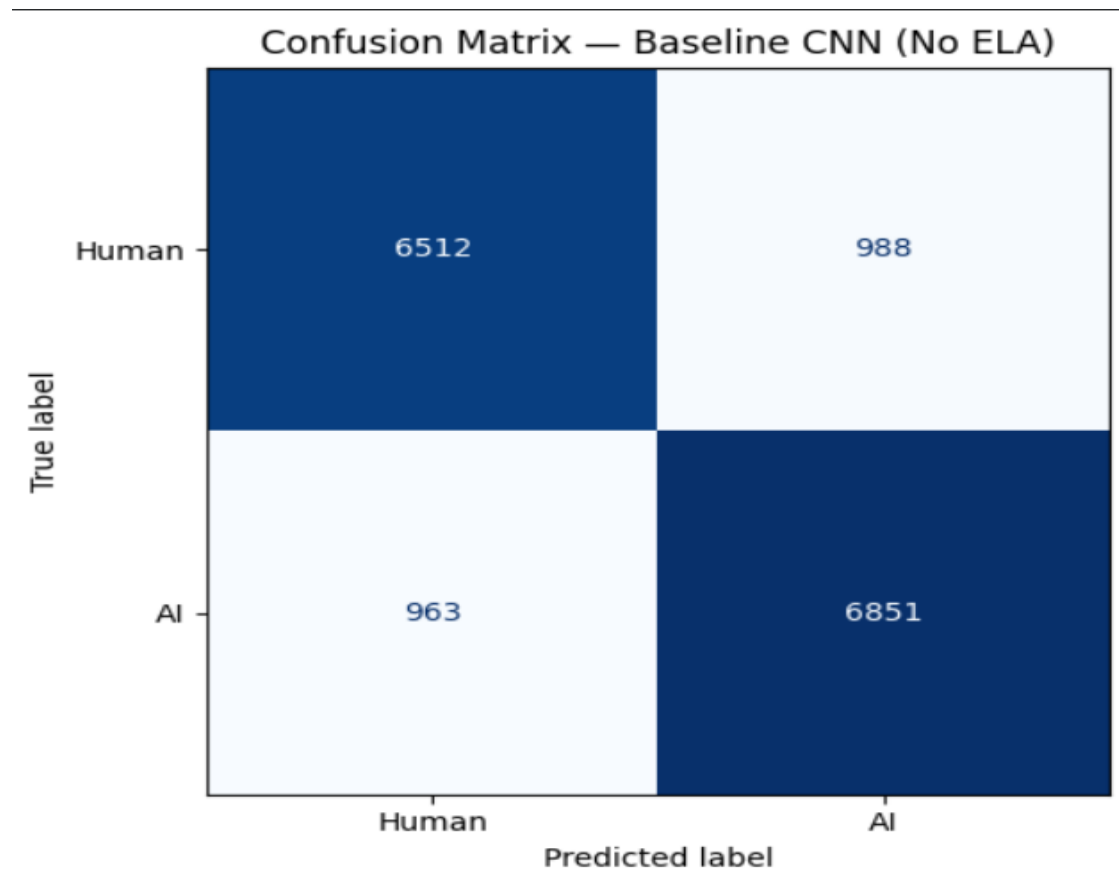


Fig. 4.8. Confusion Matrix – Baseline CNN Model

Confusion matrix indicates that the system performs reasonably well for both classes, but misclassifications are higher compared to the other models. Both False Positives and False Negatives are more prominent, reflecting limited discriminative capability.

Table 4.3 summarizes the general performance of the Baseline CNN model:

Table 4.3 Baseline CNN – Performance Metrics

Class	Precision	Recall	F1-Score	Specificity
Human	0.87	0.87	0.87	0.88
AI	0.87	0.88	0.88	0.87

4.4 COMPARATIVE ANALYSIS

Table 4.4 presents a comprehensive comparison of all three models implemented in this project. The comparison is based on overall test accuracy, average F1-score, and the computational approach used.

Model	Approach	Accuracy	Avg F1	Best Class F1
Hybrid CNN (RGB + ELA)	Deep Learning	92.43%	~0.927	0.9506 (AI)
EfficientNet-B4	Deep Learning (Transfer Learning)	93.00%	~0.880	0.95 (AI)
Baseline CNN	Deep Learning (Basic)	87.00%	~0.870	0.88 (AI)

Table 4.5 Comparative Analysis of All Four Models

4.5 EXPLAINABILITY ANALYSIS USING GRAD-CAM

Gradient weighted Class Activation Mapping (Grad-CAM) is used to enhance the interpretability of the proposed models. The technique assists in visualizing the parts of an image that make the largest contribution to the prediction of the model offering information on how the model decides.

Grad-CAM is used to create heatmaps by calculating the gradient of the target class from the final convolutional layer on the feature maps of that convolutional layer. These heatmaps highlight the most influential regions of the image that affect the output of the classification.

The visualization outcomes reveal that the model tends to focus on fine background artifacts, texture discontinuities, and compression patterns instead of the main object of the image. This is particularly evident with AI-generated images, in which artificial patterns and uneven distributions of pixels are identified.

On the contrary, in the case of human images, attention is more dispersed and naturally spread, which depicts naturalistic textures and consistency in structure. This is further improved with the incorporation of Error Level Analysis (ELA) which helps the model in detecting these artifacts and allows the model to capture those differences which cannot easily be detected by the human eye.

In general, the analysis using grad-cam proves that the model does not rely only object-level datas, but rather forensic-level features, which are effective in detecting AI-generated images. This enhances the transparency, interpretability and reliability of the system.



Fig. 4.9. Grad-CAM Visualization for AI-Generated Image

Grad-CAM determines parts of an image responsible for classifying a given image as AI-generated. The most responsible components are represented in warm colours (red and yellow) whereas the less responsible components are in cooler colours (blue).

From the illustrations, the model tends to ignore the focus of the image. Instead, the model tends to focus on areas with AI-generated images with discrepancies such as unnatural textures, inequitable lighting, and repetitive visual patterns.

The model, however, appears to ignore the focal object (the Character), but instead appears to work on fine background elements such as AI image artifacts. These background elements have fine compression artifacts and pixel disturbance artifacts that are utilized by the model, missing to the naked eye.

These findings suggest that the proposed Hybrid CNN model can learn more than just the model boundaries. Lastly, incorporating Grad-CAM provides evidence that the model does learn from AI image generation by utilizing important semantic cues, thus AI image generation model reliability and interpretability.



Fig. 4.10. Grad-CAM Visualization for Human (Real) Image

The Grad-CAM heatmap for the actual human image demonstrates the influence of image regions on the model's prediction. Red & yellow colors indicate important contributions, and blue and darker colors indicate lower contributions.

The focus of the model is given to the numerous natural features of the image such as different patterns of mountain texture, reflections of water, and lighting.

The model shows diffused attention over the number of natural elements of the image in the simplicity of the image. There are no artificial elements present in the image, which the Grad-CAM heatmap shows the model indicates.

CHAPTER 5

CONCLUSION AND FURTHER WORK

5.1 CONCLUSION

In our project, three deep learning models have been developed to differentiate between human generated images and those that have been created by a Generative Adversarial Network (GAN) and/or AI. A Hybrid CNN model using EfficientNet-B0 with Error Level Analysis (ELA) and a EfficientNet-B4 model using transfer learning on images of faces were built and tested. The results were then compared to a Baseline CNN that was trained from scratch to classify images that had been generated by AI. Through the use of feature fusion and more advanced deep learning architectures, the ability to accurately classify images has been tested and discussed.

The used dataset consists of real images and generated “artificial” images. For the real images we applied normalisation and ELA to enhance compression artefacts. The used models are a Hybrid CNN which uses both spatial features (RGB images) and forensic features (ELA images), EfficientNet-B4 which is using a pre-trained feature extraction, and a Baseline CNN as a comparison.

The results obtained from this project are summarised below.

- (a) The highest accuracy, 93%, was obtained by the EfficientNet-B4 model, which speaks to the efficacy of both transfer learning and deep feature extraction.
- (b) Hybrid CNN Model: By utilizing both RGB and ELA features, we obtained comparable accuracy of 92.43%, with more balanced classification rates.
- (c) For comparison, our simple Baseline CNN was able to achieve 87% accuracy, albeit lower than fine-tuned models. The model still showed some overfitting behavior, clearly demonstrating the value of effective feature extraction in regular neural network training.
- (d) Hybrid Approach: The Hybrid approach showed that using ELA-based forensic features improves significantly the robustness of a forensic-based model for detecting artifacts in images and videos generated by current state-of-the-art AI-based systems, and in particular for detecting very subtle artifacts.
- (e) EfficientNet-B4 learned to be biased towards the dominant class (AI images) whereas Hybrid CNN does not suffer from class-wise biased learning.
- (f) Deep learning approaches significantly outperform basic CNN-based models. Furthermore, the utilization of domain specific preprocessing (ELA) significantly improves the reliability of classification results.

In this project, we studied the integration of deep learning techniques with existing methods of forensic image analysis, with a view to detecting AI-generated images. Generative models are becoming increasingly prevalent in computer graphics, and this work represents a feasible solution to detecting such images.

5.2 LIMITATIONS

Although the system is able to perform well on a number of tests, there are a few limitations with the current system.

- (a) The model performance is sensitive to image compression levels and preprocessing settings (especially ELA quality factor), which may affect consistency when applied to images from different sources.
- (b) The models are trained, evaluated on the same datasets and performance is measured. The models are then tested on different datasets to gauge performance when the data has a different distribution.
- (c) The Baseline CNN was only trained on a subgroup of the dataset, which limits its ability to generalize effectively.
- (d) The proposed system only deals with image-level detection and does not utilize metadata or context information.
- (e) ELA detection improves detection rates but is not perfect and some sophisticated manipulations may not be detected, for instance those generated by state of art deep learning based generative models.
- (f) Models require large computational resources to train, especially for models like EfficientNet-B4.

5.3 FURTHER WORK

This project work has informed possible future research directions:

Integration of Larger and Diverse Datasets: The output of this work could be used to train and test on a larger dataset. In addition to using images of people with PWS, models could be evaluated on images of people that have been generated by other forms of AI i.e. other types of deep learning-based image generation (e.g. other types of GANs, a diffusion model).

Advanced Feature Fusion Techniques: In the current stage, simple feature concatenation is employed. It may be worth exploring more sophisticated fusion of RGB and ELA features, such as attention-based fusion or more elaborate multi-modal learning approaches.

Real-Time Detection System: This model can be trained to work in a deep learning model and deployed as a real-time application or API that quickly identifies whether an image has been generated by AI, and can be very useful for both social media monitoring and cybersecurity use cases.

Improving Model Explainability: These methods can be improved to better explain deep learning decisions, for increased trust and interpretability. (e.g. Grad-CAM, Score-CAM)

Class Imbalance Handling: We would like to investigate the use of weighted-loss functions and data augmentation in future work.

Lightweight Model Development: As with many applications, developing optimized and lightweight models that can run on mobile or edge computing devices enables more practical usage of models for various use cases.

Multi-Modal Detection: Future versions of Multi-Modal Detection will not only focus on detecting highly realistic, state-of-the-art generated “deepfakes” by image features, but will also incorporate detection by text, metadata and source verification features.

CHAPTER 6

REFERENCES

- [1] Bird J J and Lotfi A (2023). CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images. *arXiv preprint arXiv:2303.14126*.
- [2] Tan M and Le Q (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *Proceedings of the 36th International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/1905.11946>
- [3] He K, Zhang X, Ren S and Sun J (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. <https://arxiv.org/abs/1512.03385>
- [4] Simonyan K and Zisserman A (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1409.1556>
- [5] Chollet F (2017). Xception: Deep Learning with Depthwise Separable Convolutions. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. <https://arxiv.org/abs/1610.02357>
- [6] Selvaraju R R, Cogswell M, Das A, Vedantam R, Parikh D and Batra D (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. <https://arxiv.org/abs/1610.02391>
- [7] Wang H, Wang Z, Du M, Yang F, Zhang Z, Ding S, Mardziel P and Hu X (2020). Score-CAM: Score-Weighted Visual Explanations for Convolutional Neural Networks. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. <https://arxiv.org/abs/1910.01279>
- [8] Krizhevsky A, Sutskever I and Hinton G E (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [9] Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A and Bengio Y (2014). Generative Adversarial Nets. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [10] Rössler A, Cozzolino D, Verdoliva L, Riess C, Thies J and Nießner M (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.
- [11] Bayar B and Stamm M C (2016). A Deep Learning Approach to Universal Image Manipulation Detection Using a New Convolutional Layer. *Proceedings of the ACM Workshop on Information Hiding and Multimedia Security*.
- [12] Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, Huang Z, Karpathy A, Khosla A, Bernstein M, Berg A C and Fei-Fei L (2015). ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115(3), 211–252.

CHAPTER 7

APPENDIX

7.1 BASE PAPER


Multidisciplinary | Rapid Review | Open Access Journal

Received 2 July 2025, accepted 29 July 2025, date of publication 7 August 2025, date of current version 15 August 2025.
Digital Object Identifier 10.1109/ACCESS.2025.1396682

 **APPLIED RESEARCH**

Deepfakes in Visual Art: Differentiating AI-Generated Art From Human Art Using Convolutional Neural Networks (CNN)

**NGONIDZASHE TINAGO¹, SILAS FORMUNYUY VERKIJKA¹,
AND KELIBONE EVA MAMABOLO²**
Department of Computer Science and Information Technology (CSIT), Sol Plaatje University, Kimberley 8300, South Africa
Centre for Applied Data Science (CADS), Sol Plaatje University, Kimberley 8300, South Africa
Corresponding author: Silas Formunyuy Verkijka (vekasif@gmail.com)

ABSTRACT As AI technology evolves, seeing is not believing. The boundary between human and machine creativity is increasingly blurred, presenting challenges for the art industry. This is more pronounced nowadays as advancements in AI technology make it increasingly easy to create highly realistic synthetic art. This study explores the use of Convolutional Neural Networks (CNNs) to differentiate AI-generated art from human-created art. By employing Error Level Analysis (ELA), an image forensic technique for detecting fake and real images, this study develops a robust CNN classifier. Using the AI-ArtBench dataset, the optimal model achieves a 99% classification accuracy, even when tested on art from a different generative model. While AI-image detection remains a “cat and mouse” pursuit due to advancements in generative AI, the findings of this study highlight that there are clear, discriminable differences between AI-generated and human-created art. The implications of this research extend beyond academic inquiry. They offer support for artists, collectors, curators, and policymakers as they navigate the complexities of AI’s expanding availability and address the evolving role of AI in the art world. It sets the groundwork for application within fields faced with the same or similar challenges.

INDEX TERMS Artificial intelligence (AI), convolutional neural network (CNN), deep learning, diffusion model, error level analysis (ELA), generative adversarial network (GAN).

1. INTRODUCTION

Historically, photographic images were regarded as the most reliable form of evidence due to the challenges associated with altering them [1]. However, the emergence of advanced image editing software has significantly undermined this perception. Furthermore, advances in Artificial Intelligence (AI) introduce a new challenge in discerning authenticity among visually indistinguishable images produced by Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), and Diffusion Models (DM) [1]. These advances in AI have made sophisticated digital art creation and manipulation readily accessible through the introduction of Deepfakes generated using transformer architectures in conjunction with

conversational engines [2]. These technologies include, but are not limited to, Stable Diffusion, DALL·E, OpenArt, Adobe Firefly, and ImagineArt. These technologies have lowered the bar for creating deepfakes for misinformation [3], including creating art from natural language prompts.

First introduced by Goodfellow and colleagues in 2014 [4], GANs have since evolved into Creative Adversarial Networks (CANs), capable of producing realistic artworks that deceive viewers into believing they are created by professional artists. When used in collaboration with the artist, GANs could improve creativity [5], but the result of the collaboration raises questions about the role of human creativity. As a result of the widespread acceptance of AI-based image generators, China has observed a 70% drop in illustrator jobs in the gaming industry [6]. Consequently, some artists have found their artworks in a sample of training data without having

The associate editor coordinating the review of this manuscript and approving it for publication was Junchi Yan¹.

141484

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see <https://creativecommons.org/licenses/by/4.0/>

VOLUME 13, 2025

been compensated [7], [8]. Furthermore, famous paintings and photos have been replicated by image generators such as Stable Diffusion [9]. Ignoring these concerns would only encourage the marginalisation of artists.

To prevent such misuse, deepfake technology and AI content generators must be carefully regulated [10]. The generative AI industry, valued at an estimated \$48 billion [9], has raised concerns about companies like Stability AI using artists' works without compensation to compete in the \$65 billion [11] contemporary art market. The consequence runs beyond financial loss, encompassing issues such as digital artwork forgery, data laundering, and the moral rights of derived work [5], [9], [12]. To mitigate these consequences, there is a need to develop reliable systems that can distinguish human-drawn art from AI-generated art. Current work in image forensics [13] focuses on the classification of human attributes in pictures and videos. Certain artefacts, discernible from the contours of fingers, can sometimes easily be spotted within representational imagery [14]. However, this becomes increasingly challenging when dealing with landscapes, abstract objects, and similar subjects.

Consequently, researchers have explored the possibility of using deep learning techniques such as Convolutional Neural Networks (CNN) for identifying AI-generated art. However, advances in deepfakes pose a challenge to the direct application of CNN due to the lack of distinguishable objects, making the direct use of CNN less effective [2], [14]. While the direct use of a CNN may be inadequate, ELA holds promise [14]. This study aimed to utilise ELA with CNNs to differentiate and classify AI-generated art and human-created art. We consider human-created art as any visual work produced through human creativity and skill, expressed across distinct artistic styles such as realism, impressionism, cubism, and others. These works are created using mediums like painting, drawing, sculpture, or photography, and exclude those generated primarily by autonomous machine processes where the human serves only as an operator.

The CNN built for the classification task is trained on two subsets of the data: firstly, on the original data and secondly, on the ELA-enhanced data subset to establish a comparison of the effect of ELA on the model performance. Furthermore, the model's generalisation ability is evaluated by conducting a test of the model's performance on a subset of the data produced by a different generative model not seen during training.

The subsequent parts of this paper are as follows: Section II discusses existing and proposed methods for detecting AI-generated content. Following this, Section III presents the methodology used in this study, while Section IV presents the experiment and results. Section V presents a discussion of the findings and their implications before the work presented in this paper is concluded in Section VI.

II. RELATED WORK

Significant research has been conducted on detecting deepfakes—digitally manipulated images—and AI-generated

images produced by Generative Adversarial Networks (GANs). However, due to the relatively recent emergence of diffusion models, research on detecting diffusion model-based AI-generated images is still limited. As such, the subsequent related studies cover AI image detection in a broader sense, focusing on GAN AI-generated images.

A. EVOLUTION OF DIGITAL IMAGE MANIPULATION

Deepfakes date back to 1860 when John Calhoun's head was replaced with that of the U.S. President for propaganda [15]. While early deepfakes relied on manual editing [16], advances in deep learning and computer graphics now enable automated, realistic creations. The first internet deepfake emerged in September 2017, which saw face swaps of multiple famous actresses onto pornographic material [17]. Digital image manipulation has surged, driven by the widespread availability of software like Photoshop [2] and the influence of AI and machine learning. The popularity of deepfakes was fuelled by tools like deepNude for creating fake nude images [17], while generative AI tools like DALL-E 2 and Stable Diffusion [18] have enabled the creation of new images. While image and video manipulation have been practised for years in the entertainment industry, it is increasingly becoming available to a wider audience, even without any computer programming skills [17]. Therefore, there is a pressing need to develop methods that can effectively differentiate between AI-generated art images and those created by humans.

B. OVERVIEW OF AI-GENERATED IMAGE DETECTION

Research on AI-generated image detection has advanced considerably [1], with a significant number of studies concentrating on deepfakes, particularly involving facial landmarks and photorealistic content [2], [15], [19], whereas others focus on GAN-generated images [20]. One study demonstrates that human experts can achieve up to 83% accuracy in detecting AI-generated art, with greater success in identifying genres like anime and cartoons compared to more complex styles such as oil or acrylic [21]. Much of the prior work on deepfakes concentrates on facial landmarks and photorealistic content. For example, methods using mean neuron coverage (MNC) and a combination of CNNs have demonstrated varying levels of success on specific datasets [16], [22]. Additionally, CNNs and photo forensic techniques like ELA and PRNU have been employed to detect deepfakes with accuracies of up to 93% [2], [15], [23]. Despite these advancements, most techniques are for facial recognition, leaving a gap in the detection of abstract objects.

C. TECHNIQUES FOR THE DETECTION OF AI-GENERATED IMAGES

AI-generated image detection has been explored through various techniques, each with distinct strengths and limitations. Some of these techniques are discussed below.

1) DETECTION FROM HANDCRAFTED FEATURES

Early methods focused on spatial domain cues like colour, saturation, and reflections, but these methods struggle to generalise as generative models improve [24]. A common approach is the use of handcrafted features, which rely on visual inspection of the image or prior knowledge of the generative model architecture's weaknesses [25]. The visual inspection often includes detecting issues like facial asymmetry, misplaced landmarks (e.g., nose), and colour saturation inconsistencies [21], [25]. However, as generative models produce increasingly realistic images, these cues become less detectable [10]. More recent methods exploit shortcomings of the generative models used. These include Convolutional Traces and Out-of-Distribution (OOD) detection. Convolutional traces extract unique activation patterns from convolutional layers, acting as a fingerprint to identify deepfakes and the specific GAN architecture used [2], while OOD classifies images as "real" or "fake" by identifying those that fall outside the training distribution [26]. While both techniques are promising, they are model architecture-specific; hence, their effectiveness is limited to the generative model and the extent of the training data used.

2) DETECTION FROM SPATIAL DOMAIN FEATURES

Another group of methods operates in the spatial domain, exploiting low-level statistics that can differentiate real images from AI-generated ones [18], [26]. Photo Response Non-Uniformity (PRNU) is a type of noise used for source camera identification to distinguish which camera captured a particular image, achieving up to 95% test accuracy [25]. Meanwhile, patchwise feature extraction divides an image into smaller patches to identify subtle inconsistencies, such as unnatural textures or artefacts introduced by generative models, performing up to 86.88% accuracy [26]. Another such method is ELA, which analyses compression artefacts and can detect discrepancies in the image [18].

3) DETECTION FROM DATA-DRIVEN FEATURES

While earlier methods mainly focus on visually interpretable features such as colour anomalies and facial asymmetries [27], recent advances benefit from Machine Learning (ML) methods. In the context of deepfakes, particularly AI-generated, ensemble regression trees have been used to exploit visually apparent features by identifying inconsistencies in key facial landmarks such as the nose, eyebrows, jawline, mouth and eyes [28]. One study achieved a classification accuracy of 95.50% using various pre-trained CNN architectures, including a 3D ResNeXt (with non-local) classifier [25]. However, they still face limitations in generalisation. A notable vulnerability is their susceptibility to adversarial perturbations, where carefully crafted noise is introduced to intentionally mislead the CNN classifier [25]. Furthermore, the success of CNNs is highly correlated with the quality and size of the training data used. Any bias present in the dataset is often reflected in the model's predictions [29].

D. MECHANISMS OF IMPROVING CNNs

Due to the dominance of GANs in image synthesis, many of the weaknesses exploited are for GAN-based AI detectors. Rather than training the CNNs on the raw pixels, [25] notes that the GAN pipeline left behind a spectrum artefact (checkerboard artefact) that acts as a fingerprint in every image, making it possible to not only determine whether the image is fake or not, but to determine the model that created it [25]. The CNN is trained on the spectral inputs identified via Fourier transformation [30], achieving a 95% classification accuracy [25]. However, this upsampling weakness described is only available on GAN models.

To address this concern for diffusion models, PRNU and ELA are two pixel-wise feature extraction methods that are emerging. ELA is used as a pixel-wise feature extraction technique that expects a uniform error level for original images and a non-uniform error level for fake images because the compression ratio of the two is always different [15]. One study achieved 89.5% accuracy in deepfake detection by preprocessing with ELA, extracting features using ResNet, and classifying with KNN [15], while a similar approach using ShuffleNet and KNN reached 88.2% [13]. For detecting AI-generated images from diffusion models, a CNN achieved 98% accuracy [18].

III. MATERIALS AND METHODS

This section outlines the methodology used in this study, which follows the Cross-Industry Standard Process for Data Mining (CRISP-DM). This includes the data collection, pre-processing techniques and the algorithm's architecture used. Lastly, the evaluation metrics used to check the performance of the model are presented.

A. DATASET

This study uses the AI-ArtBench dataset [27], an extension of the ArtBench-10 dataset [31] that includes AI-generated art in addition to human-drawn art. It has three main categories: human-drawn art, latent diffusion-generated art and standard diffusion-generated art. The dataset contains 60,000 human-drawn art that comprise paintings, murals, and sculptures—spanning 10 distinct artistic styles from the 14th to the 21st century [31]. Fig. 1 present sample images of human-drawn art.



FIGURE 1. Sample images from the dataset showing human art.

Of the 60,000 human-drawn art images, the generative models were used to generate an additional 125,015 images to make a total of 185,015 images [23]. In image synthesis, latent and standard diffusion are generative models that iteratively add noise to data and then reverse the process to create new images [20]. They differ in that latent diffusion performs this process in a compressed, lower-dimensional latent space, while standard diffusion models operate directly in the high-dimensional data space [20]. Sample AI-generated images from the dataset are shown in Fig. 2 and Fig. 3.

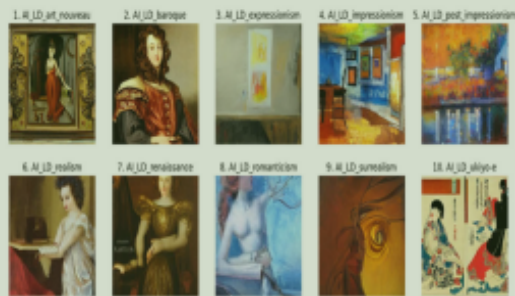


FIGURE 2. A sample of the dataset showing latent diffusion AI-generated art.



FIGURE 3. A sample of the dataset showing standard diffusion-AI-generated art.

B. IMAGE CLASSIFICATION

1) IMAGE PREPROCESSING

Data preprocessing is a technique applied to the raw data to prepare it for further processing and subsequent analysis [32]. It has a significant impact on the overall performance of machine learning algorithms, particularly Artificial Neural Networks like CNNs [32]. Common techniques include data standardisation, such as resizing images to a standard size, normalising pixel values (statistical or min-max standardisation) [32] and applying data augmentation (e.g., rotation, flipping, zooming) to artificially expand the dataset. Additionally, noise filtering can be performed, such as through a Gaussian blur, to filter out unnecessary details from the image pixels [33]. Other methods like edge

detection, histogram modification [33], and class imbalance handling (e.g., oversampling) can further enhance feature extraction and model robustness. Another such technique is class balancing, often through under- and oversampling. Oversampling aims to artificially increase the size of the underrepresented group, while undersampling removes from the overrepresented category [25].

2) FEATURE EXTRACTION

CNNs excel at automatically extracting features. However, as encouraged by the CRISP-DM methodology, it is essential to complement this process with a thorough analysis of possible features. Feature extraction from images aims to extract a brief yet informative set of features from the raw pixels to effectively capture the distinguishing characteristics among various classes or categories [34]. These features can be visible (such as uneven and unnatural facial traits) or invisible (like pixel-level statistical inconsistencies) [18]. One such method used is global histogram analysis (GHA), which represents the joint probability of intensity values across multiple colour channels. This method is translation, scale, and rotation invariant, which makes it a robust technique for machine-learning image tasks [35]. The choice of latent diffusion is motivated by the generative process of latent diffusion, which compresses the image into a lower-dimensional latent space [20], aligning well with the compression techniques used in ELA.

Another feature extraction technique used is ELA, which is a pixel-based feature extraction method used for detecting any form of image manipulation by identifying image segments with varying compression levels [36]. The ELA is obtained by taking the difference between the original image (say O_i) and an image (I_i) compressed at a known quality level [3], as seen in (1).

$$ELA = O_i - I_i \quad (1)$$

The differences highlighted by this process pinpoint areas in the image where compression has caused the most visual degradation, which is referred to as the “error level.” Images generated by AI often exhibit unique patterns in these error levels compared to human-created images, due to differences in how AI algorithms synthesise visual content [18]. These images serve as the input data source for the primary CNN used.

3) CONVOLUTIONAL NEURAL NETWORK

CNNs are algorithms popular in Computer Vision that have been effectively used in computer vision applications like classification because of their cutting-edge performance in automatically extracting from the dataset, both low-level and high-level features [15]. The network can learn significant features from the input data in its raw format, eventually lessening any necessity for labour-intensive feature extraction or extensive preprocessing steps [37]. This ability not only simplifies the overall workflow but also makes CNNs more adaptable to different types of visual data, like images of AI-generated and human-created art [15].

The proposed work trained two CNNs to learn to classify an art image into either AI-generated or human-created art. The architecture of CNNs is well-suited for image classification tasks due to its ability to automatically learn spatial hierarchies of features [38]. This is achieved through a series of operations defined by the mathematical model in (2) [39], where f represents the input image, while k is the kernel function.

$$y_{m,n} = f_{m,n} * k_{m,n} = \sum_i \sum_j f_{i,j} \cdot k_{m-i, n-j} \quad (2)$$

This model architecture consists of three main types of layers: convolution, pooling and fully connected layers. The first two – convolutional and pooling – are important for feature extraction, while the fully connected layer is responsible for feature classification [40]. In addition to the three types of layers mentioned, other significant factors include the input layer, dropout layer and activation functions. The input layer resizes the input image, preparing it for further analysis. Following this, the convolutional layers apply a series of filters to detect patterns in the images and extract the features from the images [41]. Each convolutional layer uses an activation function such as the Rectified Linear Unit (ReLU) represented by (3) [41], where x refers to the input value to the activation function, used to introduce non-linearity.

$$ReLU(x) = \begin{cases} x, & x > 0 \\ 0, & x < 0 \end{cases} \quad (3)$$

These convolutional layers are interspersed with max-pooling layers, which down-sample the feature maps. This process not only reduces the feature dimension but also mitigates noise while reducing the number of parameters. Once the feature extraction is complete, these features are flattened into a one-dimensional vector. This vector is then passed into one or more fully connected (dense) layers that facilitate the final classification. The output layer applies a softmax activation function, seen in (4) [40] to generate probabilities for the classes being predicted.

$$S(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}} \quad (4)$$

4) MODEL TRAINING

In the models' training, forward propagation is used to learn, while a loss function, such as categorical cross-entropy, measures the difference between the predicted probabilities and true labels [25]. This allows the model weights to be updated through backpropagation at each epoch, gradually refining the model's predictions [40]. The categorical cross-entropy loss function is not only useful for updating the models' weights during training, but also for evaluating the model after each epoch to monitor improvements [41]. It is defined by the mathematical equation (5), where $\hat{y}$ is the predicted label while y_m is the actual target label being predicted for sample m among a total of M samples [41].

$$H = -H = - \sum_{m=1}^M y_m \cdot \log \hat{y}_m \quad (5)$$

5) EVALUATION METRICS

The trained models' performance must be examined by testing them on data not previously seen before. This is important to study the model's ability to generalise to unseen data. To conduct this test, the model makes predictions on the test data and the results of this classification are recorded using the table-like structure referred to as a confusion matrix. The confusion matrix is a 2×2 matrix [34], containing True Positive (Correctly labelled AI art image), True Negative (Correctly labelled human art image), False Positive (Incorrectly labelled AI art image), and False Negative (Incorrectly labelled human art image).

The confusion matrix is an important tool because it allows us to compute key model performance measures [34]. These include:

- Accuracy (A), which is the probability that a randomly selected instance is correctly classified, is mathematically represented by (6) [34]. This is the total number of correctly classified predictions over the total number of predictions made.
- Precision (P) is the probability that a positive prediction is correct, mathematically represented by (7) [34].
- Recall (R), which is the probability that a true positive instance is identified, mathematically represented by (8) [34].
- F1 Score, which is the harmonic mean of precision and recall, balancing both metrics, mathematically represented by (9) [34].

$$A = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

$$P = \frac{TP}{TP + FP} \quad (7)$$

$$R = \frac{TP}{TP + FN} \quad (8)$$

$$F1 = \frac{2 * P * R}{P + R} \quad (9)$$

One of the most crucial aspects of any machine learning model is its ability to generalise to new, unseen data [42]. Overfitting can lead to models that perform well on training data but poorly on unseen data. For this study, generalisation is assessed by testing the model on the data from the standard diffusion data subset, different from the latent diffusion data subset.

IV. EXPERIMENTAL ANALYSIS

A. DATASET

This data was fairly balanced in the training set with some classes exhibiting slightly more for the human subsets compared to their corresponding AI subsets, as depicted in Fig. 4.

For example, the art nouveau class has 5007 human-drawn art images, while the corresponding art style has 4992 latent diffusion AI-generated art images. Likewise, the ukiyo-e subset has 504 more images for the latent diffusion of AI-generated art than human-created art. Meanwhile, the test set was perfectly balanced for both human and

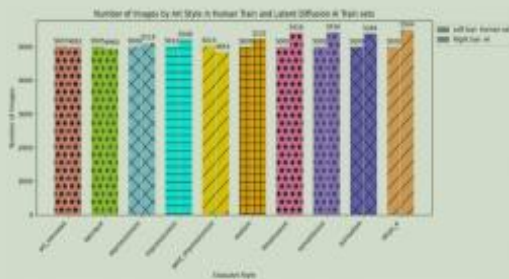


FIGURE 4. Number of images present in the training data.

AI-generated art, with each class having exactly 1000. Hence, there was no class balancing performed.

B. FEATURE EXTRACTION

1) GLOBAL HISTOGRAM ANALYSIS (GHA)

This study used global histograms to visualise pixel value distributions across the RGB colour channels for both human-drawn art and latent diffusion-generated art data subsets. Fig. 5 illustrates the global histograms for each dataset. Histogram analysis, as a feature extraction method, focuses specifically on the distribution of colour, making it particularly useful for comparing characteristics across generative models.

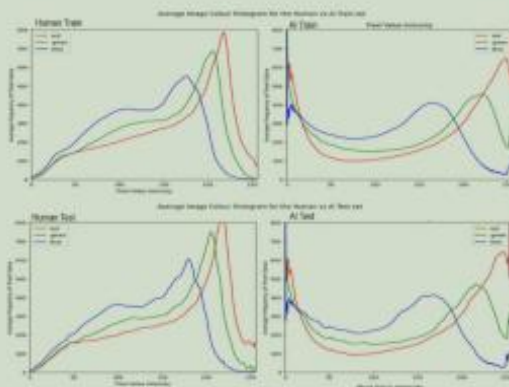


FIGURE 5. Global histograms for human and AI-generated art.

Fig. 5 shows three colour channels of pixel intensities. The distribution of pixel intensities across the human-created art seems normally distributed, and they follow the same pattern. Likewise, the AI set shows a similar pattern for train and test, which is different from the human set. By examining Fig. 5, it is evident that the patterns observed are not influenced by the set for which the histogram is produced (train or test) because both sets exhibit the same pattern for training and testing. Fig. 6 shows one sample that provides additional support for these observations.

Both colour histograms produced are histograms corresponding to the image shown on the left. The colour histogram of the AI image shown indicates that, unlike the human-created image, the colour usage in the image is biased towards brighter colours.

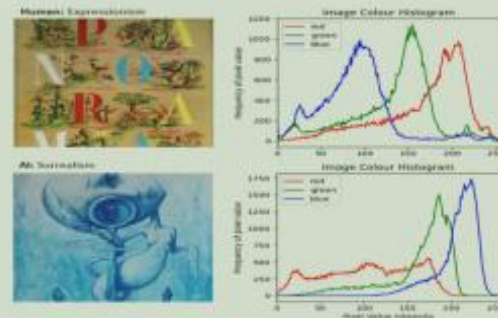


FIGURE 6. Colour histograms of human and AI-generated art.

2) PRNU

PRNU analysis shows promising potential in distinguishing AI-generated images from real ones by leveraging sensor-level noise patterns inherent to camera hardware [14], [25]. As a form of multiplicative noise, PRNU arises from manufacturing imperfections that cause pixel-level variations in light sensitivity [25]. This variation forms a unique fingerprint across images captured by the same camera. To extract this fingerprint, the image is denoised and the residual noise pattern is computed [14], [25]. Although AI-generated images do not originate from physical sensors, the application of the PRNU extraction process still yields patterns that can be used for classification [14]. Sample PRNU processed images are shown in Fig. 7, Fig. 8 and Fig. 9.



FIGURE 7. PRNU on human-created art image.

3) ELA

While histogram analysis provides insights into colour distribution across image classes, colour alone does not capture all relevant features for AI-generated art detection. Therefore, this study also incorporated ELA as a complementary feature extraction technique to evaluate the integrity of AI-generated art by examining the compression levels [18]. Using ELA, the dataset for training the primary CNN model was generated.



FIGURE 8. PRNU on a latent diffusion-generated art image.



FIGURE 9. PRNU on a standard diffusion-generated art image.

Each input image was converted from BGR to RGB to ensure accurate colour representation. A temporary JPEG version of the image was created at 90% quality to simulate a controlled lossy compression. The difference between the original and compressed images was then calculated and amplified by a factor of 10, while pixel values were kept within the acceptable range of 0 to 256 pixels. Fig. 10, Fig. 11 and Fig. 12 depict randomly sampled images produced by ELA for latent diffusion, standard diffusion and human-created art, respectively. The figure on the leftmost part indicates the original image before it underwent the ELA process to produce the noisy image on the rightmost part of the image.



FIGURE 10. ELA on Latent Diffusion generated art image.

As a collective, these ELA-enhanced images form the foundation for model training, testing, and evaluation.

C. MODEL TRAINING AND EVALUATION

In this study, two CNNs were trained and compared. The primary model was trained on the ELA-enhanced dataset, whereas a second model was trained on the original unaltered dataset that served to assess the ELA's impact



FIGURE 11. ELA on Standard Diffusion generated art image.

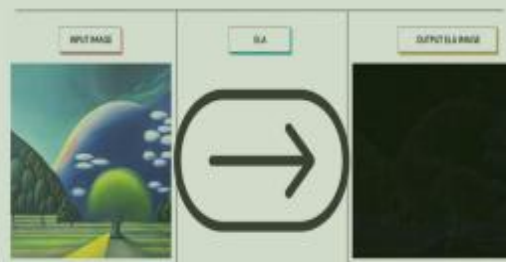


FIGURE 12. ELA on human-created art image.

on model performance. Fig. 13 presents the architecture used for both models, each implemented with a 3×3 filter size.

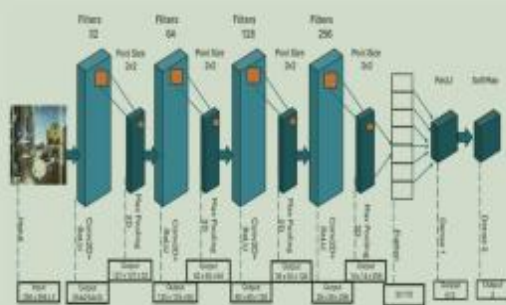


FIGURE 13. Proposed CNN model summary.

The dataset was divided into 70% for training, 15% for validation, and 15% for testing to ensure a balanced distribution for model training, hyperparameter tuning, and performance evaluation. The model described was trained over 20 epochs, compiled using the Adam optimiser with a learning rate of 0.001 and evaluated using accuracy and binary accuracy metrics. For a binary classification task, reporting binary accuracy is more meaningful when the model accuracy differs from the binary accuracy [31]. However, as shown in Fig. 14 and Fig. 15, the accuracy and binary accuracy metrics were nearly indistinguishable, so we only report the model accuracy.

Training the CNN on the ELA dataset took 6 hours and 7 minutes, significantly longer than the 1 hour and



FIGURE 14. CNN training curves on the original dataset.

56 minutes required for the unaltered dataset. During training, both models demonstrated consistent and significant improvement over the 20 epochs, achieving high training accuracies of 98.05% for the original dataset and 99.63% for the ELA-enhanced dataset. The validation accuracy for the original model reached 97.57%, while the ELA model achieved 99.06%. Upon completion of training, both models were evaluated on their respective test sets.



FIGURE 15. CNN training curves on the ELA dataset.

Despite performing similarly during training, their results diverged significantly on unseen data. While the model trained on the original dataset achieved a strong test accuracy of 92%, it was outperformed by the ELA-enhanced model, which achieved an impressive 99% test accuracy. This contrast highlights the impact of ELA preprocessing on enhancing the model's classification performance with new, unseen data, as shown Table 1. Besides just examining how well the ELA-enhanced model outperforms the standard CNN model, we performed ablation tests that replaced ELA with two of the known alternative approaches (i.e. GHA and PRNU) to validate the value of ELA in enhancing the ability of CNN to distinguish human from AI-generated art.

Our ablation results (Table 1) show that the method of feature extraction plays a critical role in distinguishing human art from AI-generated art. For latent diffusion-generated art, the baseline model is the lowest performing model (F1-score = 0.92, Accuracy = 0.92) while the model

using histogram analysis (CNN+GHA) is the best performing model (F1-score = 1.00, Accuracy = 1.00). Regarding the standard diffusion-generated art, the GHA-enhanced model underperformed all other models (F1-score = 0.49, Accuracy = 0.58) while the ELA-enhanced model was the most optimal model (F1-score = 0.99, Accuracy = 0.99). The PRNU model outperforms the base CNN model in both cases but falls short of the competition with the ELA-enhanced model. Similarly, while the ELA-enhanced model slightly underperforms the histogram analysis model for latent diffusion-generated art, the difference is minimal (less than 1%). However, the ELA-enhanced model outperforms the histogram analysis model with a wide margin when looking at the standard diffusion-generated art.

We further explored all the models using a combined dataset of latent and standard diffusion-generated art, similar to prior studies [43], [44]. The ELA-enhanced CNN model was the best-performing model (F1-score = 0.99, Accuracy = 0.99), followed by the PRNU-based CNN model (F1-score = 0.93, Accuracy = 0.93). Also observed is that the baseline CNN model achieved an F1-score of 0.80 and an Accuracy score of 0.81, outperforming the CNN model with histogram analysis (CNN+GHA), which recorded the lowest performance (F1-score = 0.78, Accuracy = 0.79). The findings indicate that the ELA-enhanced model demonstrates the most performance stability across all combinations of AI-generated images. Fig. 16, Fig. 17 and Fig. 18 show the confusion matrices for the ELA-enhanced models.



FIGURE 16. Confusion matrix - Latent diffusion generated art.

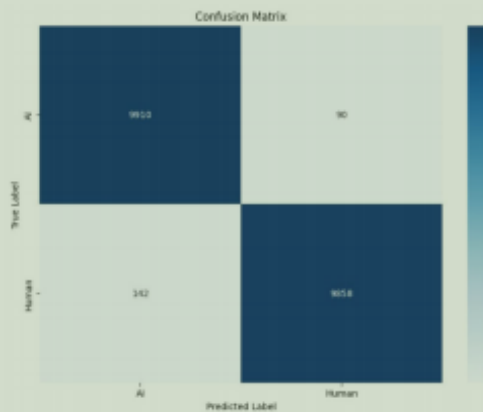
V. DISCUSSION

A. DATASET EXPLORATION: FEATURE EXTRACTION USING GLOBAL HISTOGRAM ANALYSIS

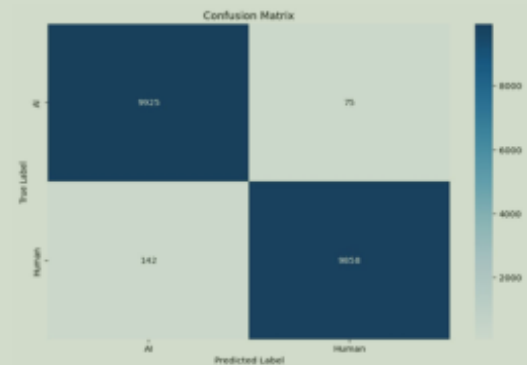
The feature extraction using analysis of the global histograms in Fig. 5 indicates that AI-generated art tends to exhibit sharp, high-contrast peaks for brighter colours such as bright red and bright green, particularly close to 256 pixels. This concentration suggests a predictable and focused colour palette in AI art. In contrast, human-made art displays broader, smoother

TABLE 1. Performance: ELA-Enhanced CNN vs. Baseline, PRNU, and Histogram Variants.

Models	Metric	Human vs Latent Diffusion Generated Art				Human vs Standard Diffusion-Generated Art				Human vs combined (Standard + Latent Diffusion) AI-Generated Art			
		AI	Human	Weighted Average	Macro Average	AI	Human	Weighted Average	Macro Average	AI	Human	Weighted Average	Macro Average
CNN (baseline)	Accuracy			0.92				0.65				0.81	
	Precision	0.89	0.95	0.92	0.92	0.86	0.59	0.72	0.72	0.92	0.74	0.83	0.83
	Recall	0.96	0.88	0.92	0.92	0.35	0.94	0.65	0.65	0.67	0.94	0.81	0.81
CNN + GHA	F1-score	0.92	0.92	0.92	0.92	0.50	0.73	0.61	0.61	0.78	0.83	0.80	0.80
	Accuracy			1.00				0.58				0.79	
	Precision	1.00	1.00	1.00	1.00	1.00	0.53	0.77	0.76	1.00	0.70	0.85	0.85
CNN+PRNU	Recall	1.00	1.00	1.00	1.00	0.16	1.00	0.58	0.58	0.58	1.00	0.79	0.79
	F1-score	1.00	1.00	1.00	1.00	0.28	0.70	0.49	0.49	0.73	0.83	0.78	0.78
	Accuracy			0.96				0.89				0.93	
CNN+ELA	Precision	0.96	0.96	0.96	0.96	0.96	0.84	0.90	0.90	0.96	0.90	0.93	0.93
	Recall	0.96	0.96	0.96	0.96	0.82	0.96	0.89	0.89	0.89	0.96	0.93	0.93
	F1-score	0.96	0.96	0.96	0.96	0.88	0.90	0.89	0.89	0.93	0.93	0.93	0.93
CNN+ELA	Accuracy			0.99				0.99				0.99	
	Precision	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99
	Recall	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99
	F1-score	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99

**FIGURE 17.** Confusion matrix - Standard diffusion-generated art.

curves, especially in the blue channel, indicating a more natural and diverse application of colour. These findings suggest that, although AI-generated artworks may appear visually similar, their underlying colour choices are more restricted. This could be attributed to the influence of the colour usage that AI image generative models observe during training. In comparison, human art reflects a richer and more balanced distribution of colours. These findings align with [24], who notes that AI image generators, particularly GANs, exhibit a limited range of colour pixel intensities. Additionally, these findings suggest that although diffusion models have become more complex than GANs, they still exhibit some shared properties that can be exploited [25]. This highlights the value

**FIGURE 18.** Confusion matrix - Combined (Standard + latent diffusion) AI-generated art.

of exploiting these visual cues when studying AI-generated images from diffusion models.

B. CLASSIFICATION RESULTS

In order to assess the value of the proposed ELA-CNN model, we compared its performance against a baseline CNN model along with alternative specifications using GHA and PRNU. All models were evaluated on three classification tasks (Table 1). The results showed that the models performed better at distinguishing AI-generated art created using latent diffusion from human-created art, compared to the other classification tasks. CNN has been widely used for image classification; however, its value in distinguishing AI-generated art from human art is less known [14], [43].

This study shows that the CNN model performs the best when focusing only on AI art generated using latent diffusion. However, it struggles when differentiating AI art generated using standard diffusion from human art, suggesting that pure CNN models might not be sufficient for detecting AI-generated art (Table 1), a view supported by [14]. A similar pattern was observed after combining the CNN model with GHA, as the model demonstrated the highest performance with latent diffusion art but performed the worst on standard diffusion art (Table 1). On the other hand, the CNN enhanced models with PRNU and ELA were more stable across the different datasets, demonstrating an adequate generalisability across latent and standard diffusion-generated art. This explains why prior studies [14], [15] have adopted mechanisms like PRNU and ELA in the classification task containing AI-generated images. The results highlight the superiority of the ELA-enhanced CNN model, which achieved near-perfect scores across all metrics—Accuracy, Precision, Recall, and F1-score—regardless of the diffusion model used to generate the AI art.

A notable characteristic shared by both models is their tendency to produce higher false positive rates—misclassifying human-created art as AI-generated—compared to false negatives, where AI-generated art is incorrectly labelled as human-made. This pattern is clearly illustrated in the confusion matrices in Fig. 13, Fig. 14, and Fig. 15. Nonetheless, the improvement in the model performance attributed to the use of ELA indicates that the direct application of a CNN model may not be the most optimal method for differentiating AI art from human art. Rather, the extraction of pixel-wise features through the ELA model yields substantially improved results. It was the only model that demonstrated consistent generalisation across all three scenarios—latent diffusion-generated art, standard diffusion-generated art, and a combination of both.

C. COMPARISON WITH EXISTING WORK

Existing research has focused extensively on detecting AI-generated images in general, particularly AI-generated faces [10]. However, there have been some recent studies [43], [44] that leverage the ArtBench-10 dataset to develop models that differentiate AI-generated art from human-created art. These studies leveraged the approach of combining the latent diffusion and standard diffusion generated art. The benchmark results are shown in Table 2.

Prior studies [43], [44] have adopted different deep learning models to differentiate AI-generated art from human art. For example, [43] proposed a CNN variant—specifically, the Attention-based ConvNeXt model—which achieved 98% accuracy in distinguishing AI-generated art from human-created art, significantly outperforming human annotators (53.8%). Similarly, [44] tested several deep learning models, demonstrating that the ResNet-50 model, run over 20 epochs, performed the best, achieving a 96.47% accuracy. This outperformed the baseline simple CNN (76.62%), a modified complex CNN (92.9%), and

TABLE 2. Comparison with Existing Models on the ArtBench-10 Dataset (Combined: Latent and Standard AI-Generated Art).

Study	Models	Test Accuracy
[43]	Human Accuracy	53.8%
	Attention-based ConvNeXt	98%
	Simple CNN	73.62
	Complex CNN	92.9%
[44]	EfficientNet	95.24
	ResNet-50	94.39
	ResNet-50 (20)	96.47
	CNN	81%
Our study	CNN with Histogram	79%
	CNN with PRNU	93%
	CNN with ELA	99%

EfficientNet (95.24%) models. Our baseline CNN model performs better than the simple CNN used by [44] but falls short of their complex CNN, which performs similarly to our CNN with PRNU (92.9% vs 93%). This highlights the view that different CNN architectures yield varying performance outcomes in the classification of AI-generated versus human-generated art. However, our proposed ELA-enhanced CNN model outperformed all benchmark comparisons, reaching an accuracy of 99%, which is one percentage point higher than the Attention-based ConvNeXt by [43]. This boost in accuracy reflects the effectiveness of ELA pixel-level feature extraction, highlighting its ability to capture subtle differences in colour usage and other visual elements between AI-generated and human art. The results suggest that simple CNN models have limited capability to distinguish between AI and human art, underscoring the need for modified CNN variants like the ELA-enhanced CNN or the Attention-based ConvNeXt model.

VI. CONCLUSION AND FUTURE WORK

This study sought to determine whether feature extraction using ELA could boost CNN's ability to accurately differentiate and classify an artistic image as either AI-generated or human-created. To achieve this, we developed an ELA-enhanced CNN model and compared its performance with a baseline CNN model as well as two variants incorporating global histogram analysis (GHA) and Photo Response Non-Uniformity (PRNU), respectively. The proposed ELA-enhanced CNN demonstrated strong generalisation capabilities, achieving a near-perfect classification accuracy of 99% on unseen data. Notably, the model maintained its performance when tested on all three subsets of the dataset, as indicated in Table 1. All the other models showed their best performance on latent diffusion-generated art, with a significant drop in performance when tested on the subset of standard diffusion-generated art. This suggests only the ELA-enhanced CNN model could effectively learn class-distinctive features without over-relying on the specific characteristics of the AI model used to generate the images

(i.e. latent vs standard diffusion). The use of ELA allowed CNN to capture subtle pixel-level differences between the structured colour distribution in AI-generated art and the more complex, varied palette characteristic of human-created art. This pattern is consistent with [14] who found that the ELA was slightly more effective than PRNU when integrated with CNN for detecting AI-created images. Additionally, when benchmarked with results from prior studies [43], [44] including human accuracy, Attention-based ConvNeXt, simple and complex CNN models, EfficientNet, and ResNet-50 models, as shown in Table 2, our results demonstrate competitive performance.

The contributions of this study are twofold. First, the study adds to the growing literature that emphasises the value of ELA [14], [15], [23] and highlights the need for integrating it into deep learning pipelines. This provides a lightweight and effective mechanism for detecting AI-generated images that outperforms complex CNN models [43], [44]. Second, besides application in visual art, this study can be extended and applied in other domains. One such example is in media and journalism. These techniques can play a crucial role in verifying the authenticity of visual content, helping to combat the spread of deepfakes and misinformation.

While these findings are valuable, there is still room for improvement. For example, with the proliferation of AI technologies, new diffusion models are likely to emerge. Having a model that generalises like the ELA-enhanced model is valuable. However, it has only been tested on two diffusion models. Future studies need to test the ELA-enhanced model on AI-generated images using other diffusion models. Another potential avenue for future work is to improve the feature extraction process. While ELA was effective, it relies heavily on detecting pixel-level discrepancies that may be subtle in certain types of AI art. A possible direction to address this is to incorporate multi-scale feature extraction techniques, such as using texture-based or frequency-based features alongside ELA. Alternatively, mechanisms to combine techniques from different studies that have shown significant levels of performance with diverse AI-generated images, such as ResNet50 [44], attention-based ConvNeXt [43] and the ELA-enhanced CNN model proposed in this study.

REFERENCES

- [1] Q. Bammei, "Synthbuster: Towards detection of diffusion model generated images," *IEEE Open J. Signal Process.*, vol. 5, pp. 1–9, 2024, doi: 10.1109/OJSP.2023.3337714.
- [2] L. Guarrera, O. Giudice, and S. Battiato, "Fighting deepfake by exposing the convolutional traces on images," *IEEE Access*, vol. 8, pp. 165085–165098, 2020, doi: 10.1109/ACCESS.2020.3023037.
- [3] B. Cavia, E. Horwitz, T. Reiss, and Y. Hoshen, "Real-time deepfake detection in the real-world," 2024, *arXiv:2406.09398*.
- [4] M. Ragot, N. Martin, and S. Cojean, "AI-generated vs. human artworks. A perception bias towards artificial intelligence?" in *Proc. Extended Abstr. CHI Conf. Hum. Factors Comput. Syst.*, Apr. 2020, pp. 1–10, doi: 10.1145/3334480.3382892.
- [5] J. Ploennigs and M. Berger, "AI art in architecture," *AI Civil Eng.*, vol. 2, no. 1, p. 8, Aug. 2023, doi: 10.1007/s43503-023-00018-y.
- [6] *AI is Already Taking Video Game Illustrators' Jobs in China*. Accessed: Mar. 22, 2024. [Online]. Available: <https://thestofworld.org/2023/ai-image-china-video-game-layoffs/>
- [7] *Claire Hummel on Twitter: '@Kelly_McKernan An Additional Resource for Searching Stable Diffusion's Training Set! I'm Not on the Lists, but a Bunch of my Stuff is Still in There: Htps://t.co/0Tx8wKIPc'/Twitter*. Accessed: Mar. 22, 2024. [Online]. Available: <https://web.archive.org/web/20230117153958/https://twitter.com/shoomlah/status/1615215285526757381>
- [8] *Karen Hallion Kenney on Twitter: '@shoomlah @Kelly_McKernan Wow, Just Searched My Name Here*. Accessed: Mar. 22, 2024. [Online]. Available: <https://web.archive.org/web/20230811043246/https://twitter.com/Khallion/status/1615464905565429760>
- [9] H. H. Jiang, L. Brown, J. Cheng, M. Khan, A. Gupta, D. Workman, A. Hanna, J. Flowers, and T. Gebru, "AI art and its impact on artists," in *Proc. AAAI/ACM Conf. AI, Ethics, Soc.*, Aug. 2023, pp. 363–374, doi: 10.1145/3600211.3604681.
- [10] G. Kalaimani, G. Kavitha, S. Chinnaiyan, and S. Mylapalli, "Optimally configured generative adversarial networks to distinguish real and AI-generated human faces," *Signal. Image Video Process.*, vol. 18, no. 11, pp. 7921–7938, Nov. 2024, doi: 10.1007/s11760-024-03440-6.
- [11] C. B. Horton Jr., M. W. White, and S. S. Iyengar, "Bias against AI art can enhance perceptions of human creativity," *Sci. Rep.*, vol. 13, no. 1, p. 19001, Nov. 2023, doi: 10.1038/s41598-023-45202-3.
- [12] Z. Epstein, S. Levine, D. G. Rand, and I. Rahwan, "Who gets credit for AI-generated art?" *iScience*, vol. 23, no. 9, Sep. 2020, Art. no. 101515, doi: 10.1016/j.isci.2020.101515.
- [13] R. Rafique, M. Nawaz, H. Kibriya, and M. Masood, "DeepFake detection using error level analysis and deep learning," in *Proc. 4th Int. Conf. Comput. Inf. Sci. (ICCCIS)*, Pakistan: IEEE, Nov. 2021, pp. 1–4, doi: 10.1109/ICCCIS54243.2021.9676375.
- [14] F. Martin-Rodriguez, R. Garcia-Mojon, and M. Fernandez-Barciela, "Detection of AI-created images using pixel-wise feature extraction and convolutional neural networks," *Sensors*, vol. 23, no. 22, p. 9037, Nov. 2023, doi: 10.3390/s23229037.
- [15] R. Rafique, R. Gantassi, R. Amin, J. Frada, A. Mustapha, and A. H. Absheri, "Deep fake detection and classification using error-level analysis and deep learning," *Sci. Rep.*, vol. 13, no. 1, p. 7422, May 2023, doi: 10.1038/s41598-023-34629-3.
- [16] R. Wang, F. Juefei-Xu, L. Ma, X. Xie, Y. Huang, J. Wang, and Y. Liu, "FakeSpotter: A simple yet robust baseline for spotting AI-synthesized fake faces," 2019, *arXiv:1909.06122*.
- [17] M. Masood, M. Nawaz, K. M. Malik, A. Javed, A. Irtaza, and H. Malik, "Deepfakes generation and detection: State-of-the-art, open challenges, countermeasures, and way forward," *Appl. Intell.*, vol. 53, no. 4, pp. 3974–4026, Jun. 2022, doi: 10.1007/s10489-022-03766-z.
- [18] F. Martin-Rodriguez, R. Garcia-Mojon, and M. Fernandez-Barciela, "Detection of AI created images using pixel-wise feature extraction and convolutional neural networks," in *Proc. Sensors*, vol. 23, no. 22, 2023, p. 9037, doi: 10.3390/s23229037.
- [19] J. Langguth, K. Pogorelov, S. Brenner, P. Filuková, and D. T. Schroeder, "Don't trust your eyes: Image manipulation in the age of DeepFakes," *Frontiers Commun.*, vol. 6, May 2021, Art. no. 632317, doi: 10.3389/fcomm.2021.632317.
- [20] P. Lorenz, R. L. Durall, and J. Keuper, "Detecting images generated by deep diffusion models using their local intrinsic dimensionality," presented at the IEEE/CVF Int. Conf. Comput. Vis., Oct. 2023. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2023W/DEAD/html/Lorenz_Detecting_Images_Generated_by_Deep_Diffusion_Models_Using_Their_Local_ICCVW_2023_paper.html
- [21] A. Yoo Jeong Ha, J. Passananti, R. Bhaskar, S. Shan, R. Southern, H. Zheng, and B. Y. Zhao, "Organic or diffused: Can we distinguish human art from AI-generated images?" 2024, *arXiv:2402.03214*.
- [22] M. Taeb and H. Chi, "Comparison of deepfake detection techniques through deep learning," *J. Cybersecurity Privacy*, vol. 2, no. 1, pp. 89–106, Mar. 2022, doi: 10.3390/jcp2010007.
- [23] I. B. K. Sudiatmika, F. Rahman, T. Trisno, and S. Suyoto, "Image forgery detection using error level analysis and deep learning," *TELKOMNIKA (Telecommunication Comput. Electron. Control)*, vol. 17, no. 2, p. 653, Apr. 2019, doi: 10.12928/TELKOMNIKA.v17i2.8976.
- [24] S. Yan, O. Li, J. Cai, Y. Hao, X. Jiang, Y. Hu, and W. Xie, "A sanity check for AI-generated image detection," 2024, *arXiv:2406.19435*.
- [25] C. Rathgeb, R. Tolosana, R. Vera-Rodriguez, and C. Busch, *Handbook of Digital Face Manipulation and Detection: From DeepFakes to Morphing Attacks* (Advances in Computer Vision and Pattern Recognition). Cham, Switzerland: Springer, 2022, doi: 10.1007/978-3-030-87664-7.

- [26] F. F. Khan, D. Kim, D. Jha, Y. Mohamed, H. H. Chang, A. Elgammal, L. Elliott, and M. Elhoseiny, "AI art neural constellation: Revealing the collective and contrastive state of AI-generated and human art," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 7470–7478, doi: 10.1109/CVPRW63382.2024.00742.
- [27] *AI-ArtBench*. Accessed: Nov. 9, 2024. [Online]. Available: <https://www.kaggle.com/datasets/ravidussilva/real-ai-art>
- [28] V. Kazemi and J. Sullivan, "One millisecond face alignment with an ensemble of regression trees," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 1867–1874, doi: 10.1109/CVPR.2014.241.
- [29] E. Pntelas, I. E. Livieris, S. Kotsiantis, and P. Pntelas, "A multi-view-CNN framework for deep representation learning in image classification," *Comput. Vis. Image Understand.*, vol. 232, Jul. 2023, Art. no. 103687, doi: 10.1016/j.cviu.2023.103687.
- [30] F. Charnot, Z. Geradts, and E. Haasdijk, "Deepfake forensics: Cross-manipulation robustness of feedforward- and recurrent convolutional forgery detection methods," *Forensic Sci. Int., Digit. Invest.*, vol. 40, Mar. 2022, Art. no. 301374, doi: 10.1016/j.fsi.2022.301374.
- [31] P. Liao, X. Li, X. Liu, and K. Keutzer, "The ArtBench dataset: Benchmarking generative models with artworks," 2022, *arXiv:2206.11404*.
- [32] M. A. R. Khan, M. M. Rahman, K. Kadingama, and R. A. Bakar, "Artificial neural network model for material removal rate of Ti-15-3 in electrical discharge machining," *Energy Educ. Sci. Technol. Part A, Energy Sci. Res.*, vol. 29, no. 2, pp. 1025–1038, 2012.
- [33] B. Chitradevi and P. Srimathi, "An overview on image processing techniques," *Int. J. Innov. Res. Comput. Commun. Eng.*, vol. 2, no. 11, pp. 6466–6472, 2007.
- [34] N. Sharma, V. Jain, and A. Mishra, "An analysis of convolutional neural networks for image classification," *Proc. Comput. Sci.*, vol. 132, pp. 377–384, Jun. 2018, doi: 10.1016/j.procs.2018.05.198.
- [35] A. Banerjee, "Detecting AI-generated images created by diffusion models," Princeton Univ., Princeton, NJ, USA, Independ. Work Rep., May 2023. [Online]. Available: https://arshbanerjee.com/uploads/papers/4d50d_20230723185830.pdf
- [36] B. Yan, C.-T. Li, and X. Lu, "JRC: Deepfake detection via joint reconstruction and classification," *Neurocomputing*, vol. 598, Sep. 2024, Art. no. 127862, doi: 10.1016/j.neucom.2024.127862.
- [37] M. Galey, F. H. Al Mukthar, R. J. Maarroof, and F. Rofoof, "Deep neural network concepts for classification using convolutional neural network: A systematic review and evaluation," *Technium Romanian J. Appl. Sci. Technol.*, vol. 3, no. 8, pp. 58–70, Sep. 2021, doi: 10.47577/technium.v3i8.4554.
- [38] C. Farabet, C. Couprie, L. Najman, and Y. LeCun, "Learning hierarchical features for scene labeling," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1915–1929, Aug. 2013, doi: 10.1109/TPAMI.2012.231.
- [39] Y. Wang, Y. Hao, and A. Xu Cong, "Harnessing machine learning for discerning AI-generated synthetic images," 2024, *arXiv:2401.07358*.
- [40] M. Tripathi, "Analysis of convolutional neural network based image classification techniques," *J. Innov. Image Process.*, vol. 3, no. 2, pp. 100–117, Jun. 2021, doi: 10.36548/jiip.2021.2.003.
- [41] S. S. M. Alla and K. Athota, "Brain tumor detection using transfer learning in deep learning," *Indian J. Sci. Technol.*, vol. 15, no. 40, pp. 2093–2102, Oct. 2022, doi: 10.17485/ijst/v15i40.1307.
- [42] G. J. Simon and C. Aliferis, *Artificial Intelligence and Machine Learning in Health Care and Medical Sciences: Best Practices and Pitfalls*. Cham, Switzerland: Springer, 2024, doi: 10.1007/978-3-031-39355-6.
- [43] R. S. R. Silva, A. Lotfi, I. K. Ihianle, G. Shahtahmassebi, and J. J. Bird, "ArtBrain: An explainable end-to-end toolkit for classification and attribution of AI-generated art and style," 2024, *arXiv:2412.01512*.
- [44] M. E. S. Tenorio, *PNTING: Detecting AI in the Painting World*. [Online]. Available: <https://cs231n.stanford.edu/papers/pnting-detecting-ai-in-the-painting-world.pdf>



NGONIDZASHE TINAGO received the B.Sc. degree in data science and the B.Sc. Honors degree in data science from Sol Plaatje University, South Africa, in 2023 and 2024, respectively. His research interests include generative AI, cybersecurity, and human-computer interaction.



SILAS FORMUNYUY VERKUIKA received the B.Sc., B.Sc. Honors, M.Sc., and Ph.D. degrees in computer information systems from the University of the Free State. He conducts research in several information system-related areas (human-computer interaction, information security, e-government, e/m-commerce, human-centered artificial intelligence, and applied data science). He has published papers in some of the leading academic journals in the field (e.g., *Computers and Security*, *Computers and Education*, *Computers in Human Behavior*, *International Journal of Information Management*, and *Journal of Retailing and Consumer Services*). He is an NRF-Rated Researcher and listed among the world's top 2% most-cited scientists by the Stanford-Elservier ranking, in 2023 and 2024.



KELIBONE EVA MAMABOLO received the B.Sc. degree in computer science from the University of Johannesburg and the M.I.T. degree in software development from Tshwane University of Technology. She is currently pursuing the Ph.D. degree in applied data science, focusing on learning analytics with the University of Johannesburg. Her research interests include learning analytics, data science, human-computer interaction, and the integration of technology in community engagement initiatives.

7.2 SAMPLE SOURCE CODE

7.2.1 Data Loading for Hybrid CNN

```
import os
train_dir = "/content/train"
test_dir = "/content/test"
def image_paths_and_labels(directory):
    image_paths = []
    labels = []

    for folder in sorted(os.listdir(directory)):
        folder_path = os.path.join(directory, folder)

        if folder.startswith("AI"):
            label = 0
        else:
            label = 1

        for img in sorted(os.listdir(folder_path)):
            image_paths.append(os.path.join(folder_path, img))
            labels.append(label)

    return image_paths, labels
train_paths, train_labels = image_paths_and_labels(train_dir)
test_paths, test_labels = image_paths_and_labels(test_dir)

print("Train images:", len(train_paths))
print("Test images:", len(test_paths))
```

7.2.2 Dataset Balancing

```
#dataset balancing
def balance_dataset(paths, labels):

    paths_ai = [p for p, l in zip(paths, labels) if l == 0]
    paths_human = [p for p, l in zip(paths, labels) if l == 1]

    min_count = min(len(paths_ai), len(paths_human))

    paths_ai = random.sample(paths_ai, min_count)
    paths_human = random.sample(paths_human, min_count)

    new_paths = paths_ai + paths_human
    new_labels = [0]*min_count + [1]*min_count

    combined = list(zip(new_paths, new_labels))
    random.shuffle(combined)

    new_paths, new_labels = zip(*combined)

    return list(new_paths), list(new_labels)

# balance train
train_paths, train_labels = balance_dataset(train_paths, train_labels)

# balance validation
val_paths, val_labels = balance_dataset(val_paths, val_labels)
```

7.2.3 Image Preprocessing

```
# ===== ELA FUNCTION =====
def convert_to_ela_path(path, quality=90):

    image = Image.open(path).convert("RGB")

    buffer = io.BytesIO()
    image.save(buffer, "JPEG", quality=quality)
    buffer.seek(0)

    compressed = Image.open(buffer)

    ela_image = ImageChops.difference(image, compressed)

    extrema = ela_image.getextrema()
    max_diff = max([ex[1] for ex in extrema])

    scale = 255.0 / max_diff if max_diff != 0 else 1

    ela_image = ImageEnhance.Brightness(ela_image).enhance(scale)

    ela_image = ela_image.resize((IMG_SIZE, IMG_SIZE))

    ela_array = np.array(ela_image) / 255.0

    return ela_array.astype("float32")
```

7.2.4 Data Pipeline

```
# ===== LOAD RGB + ELA =====
def load_pair(path, label):

    if isinstance(path, tf.Tensor):
        path = path.numpy().decode()

    # RGB image
    img = Image.open(path).convert("RGB").resize((IMG_SIZE, IMG_SIZE))
    img = np.array(img) / 255.0

    # ELA image
    ela = convert_to_ela_path(path)

    return img.astype(np.float32), ela.astype(np.float32), label
```



```
def tf_load_pair(path, label):

    img, ela, label = tf.py_function(
        func=load_pair,
        inp=[path, label],
        Tout=[tf.float32, tf.float32, tf.int32]
    )

    img.set_shape([IMG_SIZE, IMG_SIZE, 3])
    ela.set_shape([IMG_SIZE, IMG_SIZE, 3])
    label.set_shape([])

    return (img, ela), label
```

7.2.5 Hybrid Model Architecture

```
rgb_input = layers.Input(shape=(224,224,3), name="rgb_input")

rgb_base = EfficientNetB0(
    include_top=False,
    weights="imagenet",
    input_tensor=rgb_input
)

rgb_base.trainable = False

rgb_features = layers.GlobalAveragePooling2D()(rgb_base.output)

# ===== ELA INPUT =====
ela_input = layers.Input(shape=(224,224,3), name="ela_input")

x = layers.Conv2D(32, 3, activation="relu")(ela_input)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(64, 3, activation="relu")(x)
x = layers.MaxPooling2D()(x)

ela_features = layers.GlobalAveragePooling2D()(x)

# ===== FUSION =====
combined = layers.Concatenate()([rgb_features, ela_features])

x = layers.BatchNormalization()(combined)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.5)(x)

output = layers.Dense(1, activation="sigmoid")(x)
```

7.2.6 Model Training

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.3,
    patience=3,
    min_lr=1e-6
)

```

```

history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=6,
    steps_per_epoch=150,
    validation_steps=40,
    callbacks=[early_stop, reduce_lr]
)

print("Final Training Accuracy:", history.history['accuracy'][-1])
print("Final Validation Accuracy:", history.history['val_accuracy'][-1])

```

7.2.7 Fine Tuning

```

# 1 Unfreeze top part of the model
for layer in model.layers[-120:]: # adjust 80-150 if needed
    layer.trainable = True

print("Top layers unfrozen ✅")

# 2 Recompile with LOW learning rate
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5), # very important
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

print("Model recompiled for fine-tuning ✅")

# 3 Continue training
history_ft = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=5, # small number is enough
    steps_per_epoch=150,
    validation_steps=40,
    callbacks=[early_stop, reduce_lr]
)

```

7.2.8 Data Loading for EfficientNet-B4 CNN

```
# Hyperparameters
IMG_SIZE = 224
BATCH_SIZE = 16
NUM_WORKERS = 2
SEED = 42
TRAIN_RATIO = 0.7
VAL_RATIO = 0.15
# TEST_RATIO = 0.15 (remainder)

torch.manual_seed(SEED)

# ImageNet normalization stats
MEAN = [0.485, 0.456, 0.406]
STD = [0.229, 0.224, 0.225]

# Transforms
train_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.2),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2, hue=0.1),
    transforms.RandomGrayscale(p=0.05),
    transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 2.0)),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD)
])
```

```
# Transforms
train_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.2),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2, hue=0.1),
    transforms.RandomGrayscale(p=0.05),
    transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 2.0)),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD)
])

val_test_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD)
])

# Load full dataset
full_dataset = datasets.ImageFolder(DATA_DIR)
class_names = full_dataset.classes
num_classes = len(class_names)
total = len(full_dataset)
```

7.2.9 Class Balancing

```
#class balancing
all_labels = [full_dataset.targets[i] for i in train_ds.indices]
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(all_labels),
    y=all_labels
)
class_weights_tensor = torch.tensor(class_weights, dtype=torch.float)
print(f'\n🔧 Class weights (for loss): {dict(zip(class_names, class_weights.round(3)))}')
print(f'    (Higher weight = minority class penalized more when missed)')

#class balancing
sample_weights = [class_weights[label] for label in all_labels]
sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(sample_weights),
    replacement=True
)
```

7.2.10 Model Building

```
import timm
import torch.nn as nn

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {DEVICE}')

# Build EfficientNet-B4
model = timm.create_model(
    'efficientnet_b4',
    pretrained=True,
    num_classes=num_classes
)

model = model.to(DEVICE)

# Count parameters
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'\nTotal parameters    : {total_params:,}')
print(f'Trainable parameters: {trainable_params:,}')

# Verify output shape
dummy = torch.randn(2, 3, IMG_SIZE, IMG_SIZE).to(DEVICE)
with torch.no_grad():
    out = model(dummy)
print(f'\nOutput shape: {out.shape} ← should be [2, {num_classes}]')
```

7.2.11 Training

```
# PHASE 1: Warmup (classifier head only)
print('='*60)
print('PHASE 1: Warmup - Training classifier head only')
print('='*60)

optimizer = optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=1e-3, weight_decay=1e-4
)
scheduler = CosineAnnealingLR(optimizer, T_max=WARMUP_EPOCHS, eta_min=1e-5)

best_val_f1      = 0.0
best_model_path = '/content/best_efficientnet_b4.pth'

for epoch in range(1, WARMUP_EPOCHS + 1):
    t0 = time.time()
    tr_loss, tr_acc      = train_epoch(model, train_loader, optimizer, criterion, scaler, DEVICE)
    vl_loss, vl_acc, vl_f1 = eval_epoch(model, val_loader, criterion, DEVICE)
    scheduler.step()
    elapsed = time.time() - t0
```

```
# PHASE 2: Full Fine-tuning
print('='*60)
print('PHASE 2: Full Fine-tuning - All layers unfrozen')
print('='*60)

unfreeze_all(model)

optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=FINETUNE_EPOCHS, eta_min=1e-6)

for epoch in range(WARMUP_EPOCHS + 1, TOTAL_EPOCHS + 1):
    t0 = time.time()
    tr_loss, tr_acc      = train_epoch(model, train_loader, optimizer, criterion, scaler, DEVICE)
    vl_loss, vl_acc, vl_f1 = eval_epoch(model, val_loader, criterion, DEVICE)
    scheduler.step()
    elapsed = time.time() - t0
```

7.2.12 Grad-CAM Function

```
# Target the last convolutional block of EfficientNet-B4
target_layers = [model.blocks[-2][-1].conv_pwl]

cam = GradCAMPlusPlus(model=model, target_layers=target_layers)

def generate_gradcam(model, cam, image_tensor, label, class_names, device):
    """Generate Grad-CAM++ heatmap for a single image."""
    model.eval()
    image_tensor = image_tensor.unsqueeze(0).to(device)

    with torch.no_grad():
        with autocast(device_type='cuda' if torch.cuda.is_available() else 'cpu'):
            output = model(image_tensor)
            pred_idx = output.argmax(dim=1).item()
            pred_prob = torch.softmax(output, dim=1)[0][pred_idx].item()

    targets = [ClassifierOutputTarget(pred_idx)]
    grayscale = cam(input_tensor=image_tensor, targets=targets)
    grayscale = grayscale[0, :]

    # Denormalize for visualization
    mean = torch.tensor(MEAN).view(3,1,1)
    std = torch.tensor(STD).view(3,1,1)
    rgb = torch.clamp(image_tensor[0].cpu() * std + mean, 0, 1)
    rgb = rgb.permute(1, 2, 0).numpy()

    cam_img = show_cam_on_image(rgb, grayscale, use_rgb=True)
    return rgb, cam_img, pred_idx, pred_prob
```

7.2.13 GUI Implementation

```
# — Display results —————
def show_results(pil_img, pred_label, confidence, probs, rgb_np, grayscale, cam_overlay):
    is_fake = 'fake' in pred_label.lower()
    color = '#e74c3c' if is_fake else '#2ecc71'
    verdict = '🚫 AI GENERATED' if is_fake else '✅ REAL IMAGE'

    # — Header banner —————
    display(HTML(f"""
<div style="
background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%);
border-left: 5px solid {color};
border-radius: 12px;
padding: 20px 28px;
margin: 12px 0;
font-family: 'Courier New', monospace;
">

<div style="font-size:28px; font-weight:900; color:{color}; letter-spacing:2px;">
{verdict}
</div>
<div style="font-size:14px; color:#aaa; margin-top:6px;">
Confidence: <span style="color:white; font-weight:bold;">{confidence*100:.1f}%</span>
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;& Model: <span style="color:white;">EfficientNet-B4</span>
&nbsp;&nbsp;&nbsp;& XAI: <span style="color:white;">Grad-CAM++</span>
</div>
</div>
"""))

# — Figure: Original | Heatmap | Overlay | Prob bar —————
fig, axes = plt.subplots(1, 4, figsize=(18, 4))
fig.patch.set_facecolor('#0f0f1a')

titles = ['Original', 'CAM Heatmap', 'Grad-CAM++ Overlay', 'Class Probabilities']
for ax, title in zip(axes, titles):
    ax.set_facecolor('#0f0f1a')
```

```

titles = ['Original', 'CAM Heatmap', 'Grad-CAM++ Overlay', 'Class Probabilities']
for ax, title in zip(axes, titles):
    ax.set_facecolor('#0f0f1a')
    ax.set_title(title, color='white', fontsize=11, fontweight='bold', pad=8)
    ax.tick_params(colors='white')
    for spine in ax.spines.values():
        spine.set_edgecolor('#333')

# Original
axes[0].imshow(rgb_np)
axes[0].axis('off')

# Heatmap
axes[1].imshow(heatmap, cmap='jet')
axes[1].axis('off')

# Overlay
axes[2].imshow(cam_overlay)
axes[2].axis('off')

# Probability bar chart
colors_bar = ['#e74c3c' if 'fake' in c.lower() else '#2ecc71' for c in class_names]
bars = axes[3].barh(class_names, probs * 100, color=colors_bar, height=0.5, edgecolor='white', linewidth=0.5)
axes[3].set_xlim(0, 100)
axes[3].set_xlabel('Probability (%)', color='white', fontsize=10)
axes[3].tick_params(axis='x', colors='white')
axes[3].tick_params(axis='y', colors='white')
for bar, prob in zip(bars, probs):
    axes[3].text(bar.get_width() + 1, bar.get_y() + bar.get_height()/2,
                  f'{prob*100:.1f}%', va='center', color='white', fontsize=10, fontweight='bold')

```



```

# --- Widget UI ---
title_html = HTML("""
<div style="
    background: linear-gradient(135deg, #1a1a2e, #0f3460);
    border-radius: 16px;
    padding: 24px 32px;
    margin-bottom: 16px;
    text-align: center;
    font-family: 'Courier New', monospace;
    border: 1px solid #e94560;
">
    <div style="font-size:26px; font-weight:900; color:#e94560; letter-spacing:3px;">
        🦋 AI IMAGE DETECTOR
    </div>
    <div style="font-size:13px; color:#aaa; margin-top:6px;">
        EfficientNet-B4 + Grad-CAM++ &nbsp;  |&nbsp;   Upload any image to test
    </div>
</div>
""")

upload_btn = widgets.FileUpload(accept='image/*', multiple=False,
                                description='📁 Upload Image',
                                layout=widgets.Layout(width='220px'))
predict_btn = widgets.Button(description='🦋 Analyze Image',
                              button_style='danger',
                              layout=widgets.Layout(width='180px', height='38px'))
status_out = widgets.Output()
result_out = widgets.Output()

```